

# Transformations and homographies

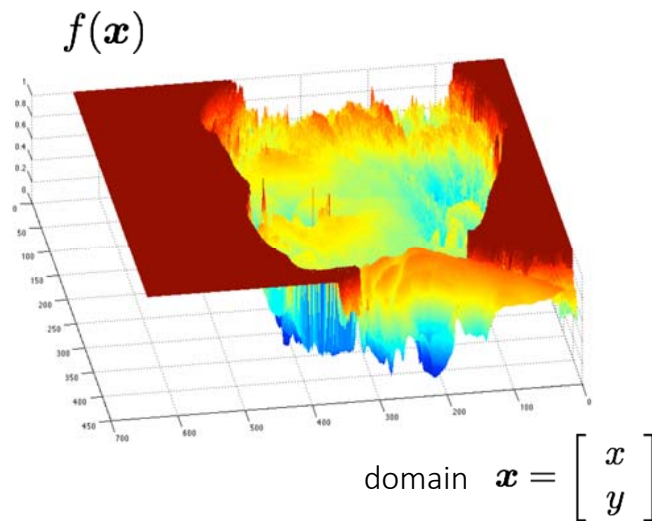


## Image transformations and warping

# What is an image?



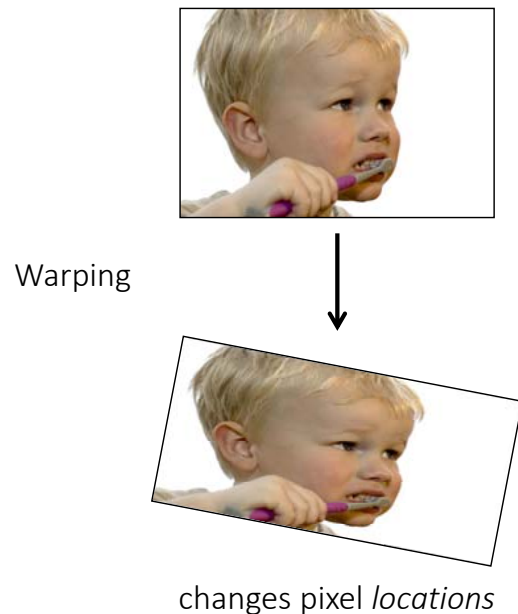
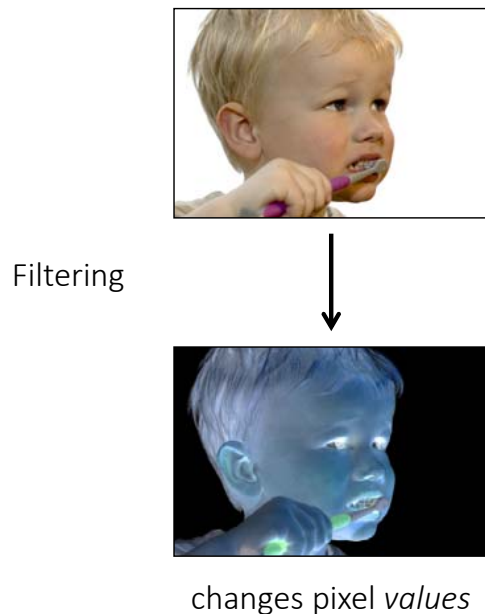
grayscale image



A (grayscale) image is a 2D function.

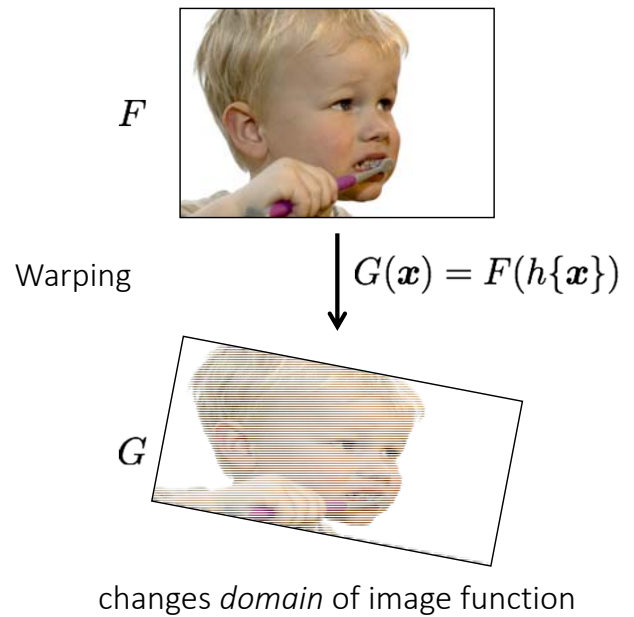
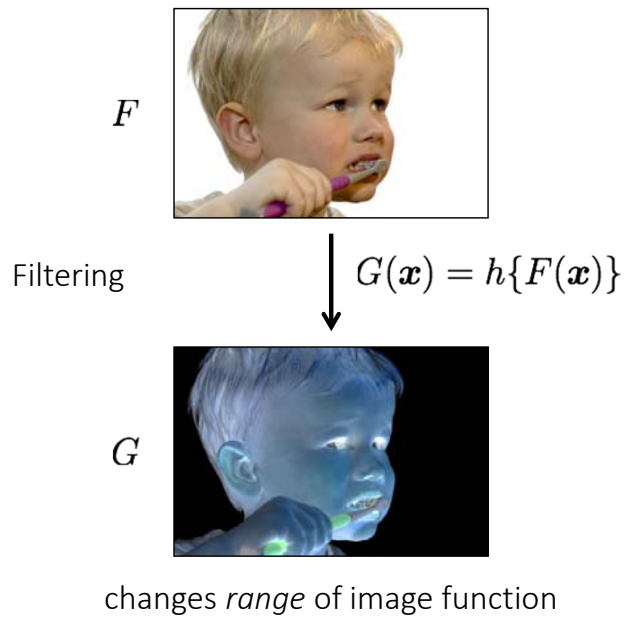
In this lecture, we will learn all about 2D transformations, warping, and homographies. Let's begin with a quick look at what we mean by warping. We often think of images as functions: The domain of these functions is the set of all 2D locations  $(x,y)$ ; and the range of these functions is the grayscale scalar intensities, or the vector RGB colors.

## What types of image transformations can we do?



With this view in mind, we can broadly think of two types of image transformations. The first type is filtering transformations, where we change the pixel values, grayscale intensities or RGB colors. These are the transformations covered in an Image Processing class. The second type is warping transformations, where we change pixel locations, while leaving pixel values the same.

## What types of image transformations can we do?



Put differently, in filtering transformations we change the range of the image function, whereas in warping transformations, we change the domain of the image function. In this lecture we will analyze warping transformations in a lot of detail.

## Warping example



As a quick preview of the things we will learn, let's say that we have these two images of the same object under a different pose. One image is a warp of the other.

## Warping example: feature matching



How do you compute the transformation?

We can use the algorithms we covered during the Feature Detection and Matching lecture, to find points that are the same in the two images. This is a very common operation in several computer vision applications, including object recognition, 3D reconstruction, augmented reality, and image stitching. Once we have these points, we can ask the question: What is the warp that maps one image to the other?

## Warping example: feature matching

Given a set of matched points...

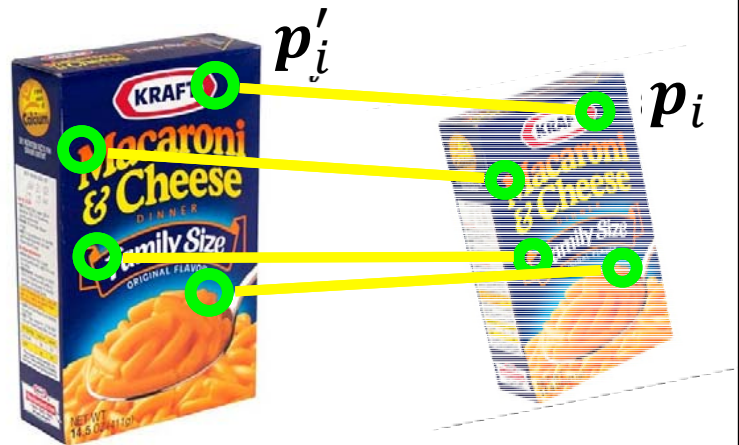
$$\{p_i, p'_i\}$$

2D point in original image    2D point in transformed image

... and a transformation:

$$p' = f(p, m)$$

transformation function    parameters



find the best estimate of the parameters  $m$

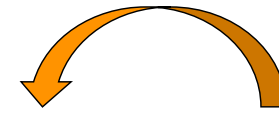
What kind of transformation functions  $f$  are there?

We will learn both how to characterize the types of warps that can produce these types of image pairs, as well as how to compute unknown warps given matched points in the image pairs.

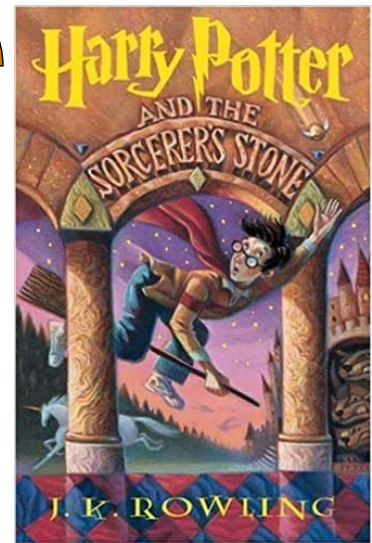
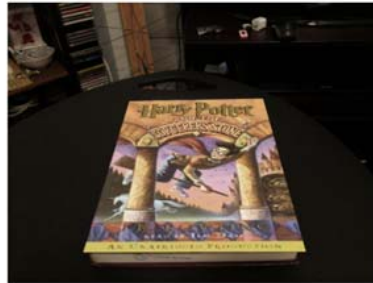


## Applications of image transformations

## Augmented reality



Compute warping transformation



# How do you create a panorama?

Panorama: an image of (near) 360° field of view.



# How do you create a panorama?

Panorama: an image of (near) 360° field of view.



1. Use a very wide-angle lens.

## Wide-angle lenses

Fish-eye lens: can produce (near) hemispherical field of view.



What are the pros and cons of this?



# How do you create a panorama?

Panorama: an image of (near) 360° field of view.



1. Use a very wide-angle lens.

- Pros: Everything is done optically, single capture.
- Cons: Lens is super expensive and bulky, lots of distortion (can be dealt-with in post).

Any alternative to this?



# How do you create a panorama?

Panorama: an image of (near) 360° field of view.



1. Use a very wide-angle lens.
  - Pros: Everything is done optically, single capture.
  - Cons: Lens is super expensive and bulky, lots of distortion (can be dealt-with in post).
2. Capture multiple images and combine them.

## Panoramas from image stitching

1. Capture multiple images from different viewpoints.



2. Stitch them together into a virtual wide-angle image.





## 2D transformations

## 2D transformations

linear transformations



translation



rotation



aspect



affine



perspective



cylindrical

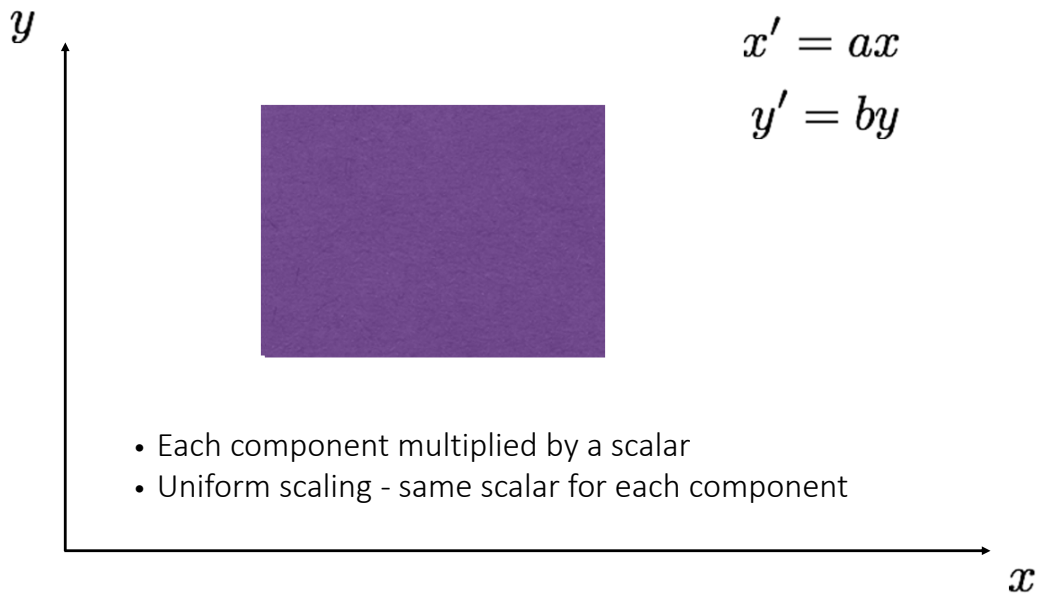
Now let's take a look at the types of warping transformations we will be covering. There are many types of warps, like the examples shown here: translating or rotating an image, changing its aspect or perspective ratio, or even distorting it based on some non-linear function. In this lecture, we will focus on two-dimensional *linear* transformations. This category includes all of the transformation examples shown here, *except* for the cylindrical one.

## 2D planar transformations



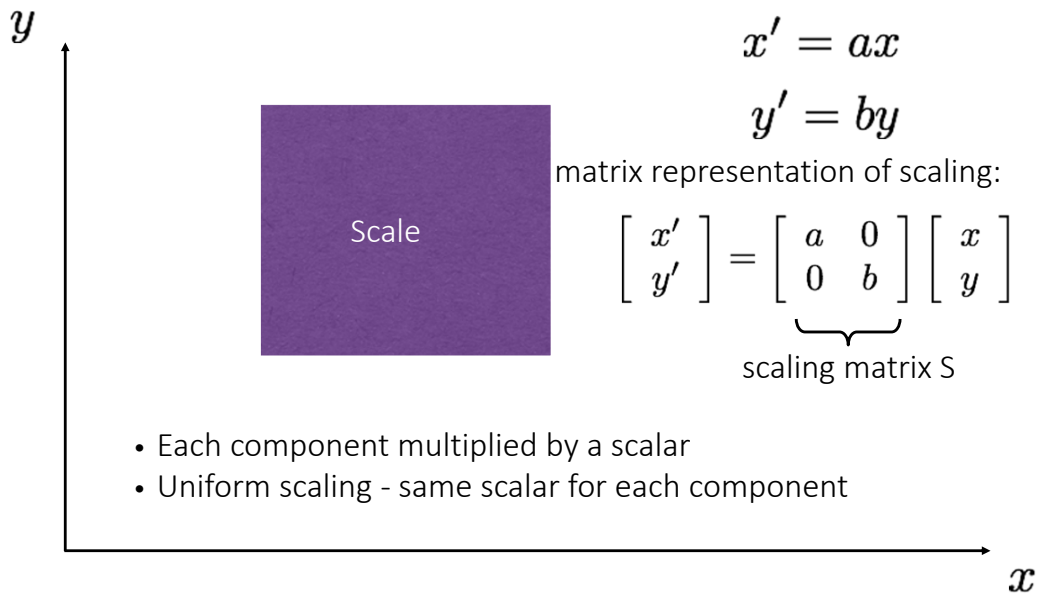
The first topic we will cover is how we can characterize these transformations mathematically. For this, let's begin with a very simple image as an example: Just a square in a white background, with the x-y axes and origin of the image domain as shown here.

## 2D planar transformations



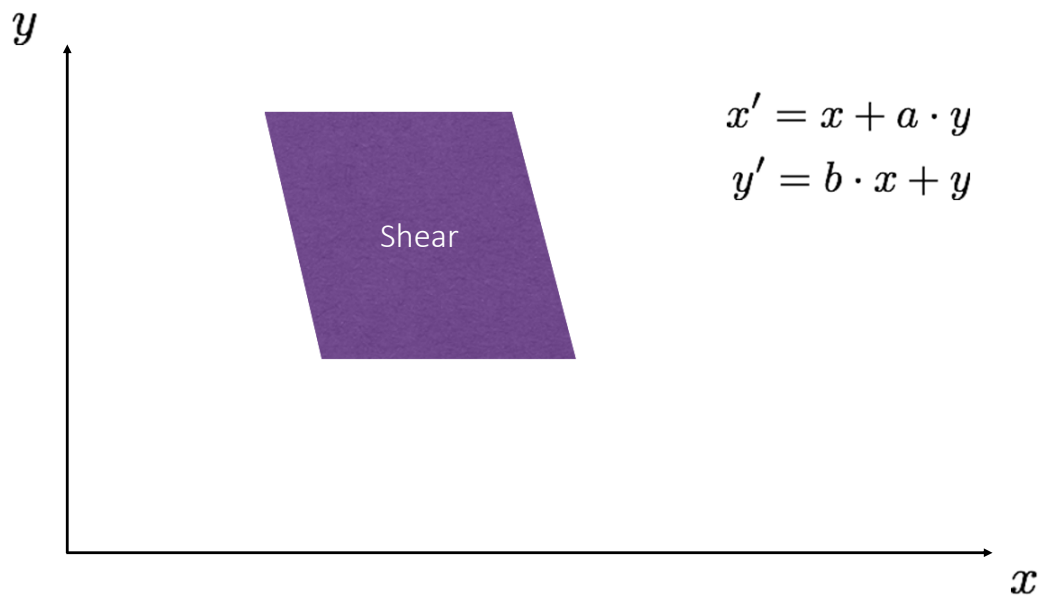
One of the simplest types of transformations is scaling. Mathematically, this means taking the  $x$ - $y$  coordinates of every image point, and multiplying them with some scalar. This scalar can be the same for both coordinates, in which case we say we have *uniform* scaling. Or, it can be different, in which case the height and width of the image change by different amounts.

## 2D planar transformations



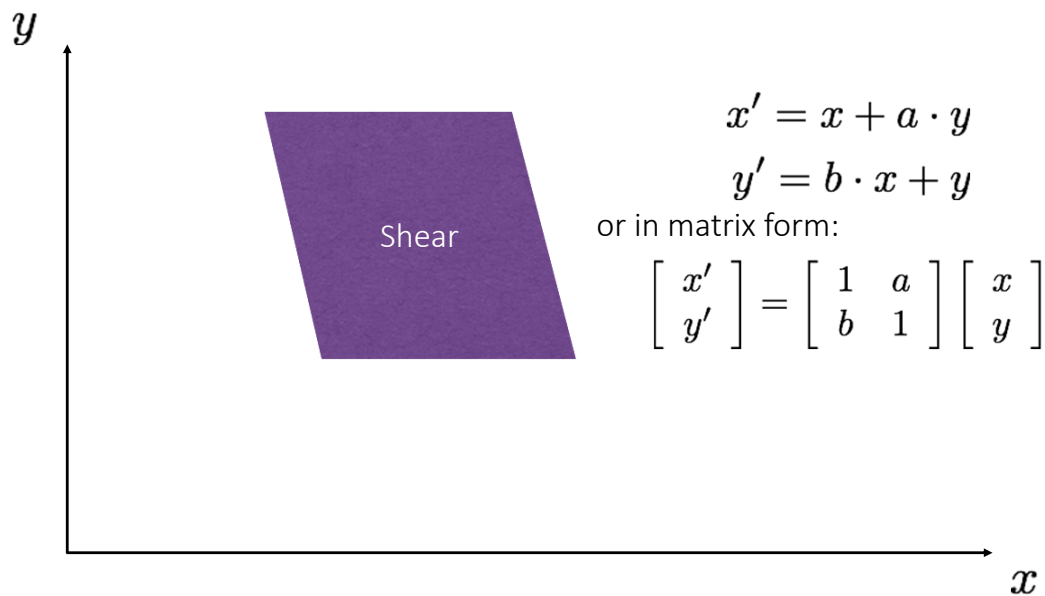
Given that we are focusing on *linear* 2D transformations, throughout this lecture, we will find it convenient to describe these transformations as matrices, multiplied by 2D x-y vectors, to produce the new warped 2D vectors. In the case of scaling, this gives us a diagonal scaling matrix, where the diagonal entries are the scaling factors for x and y.

## 2D planar transformations



Here is another type of transformation: a shear. In this case, we add to each coordinate a scaled version of the other coordinate, in order to stretch our image in one direction or another.

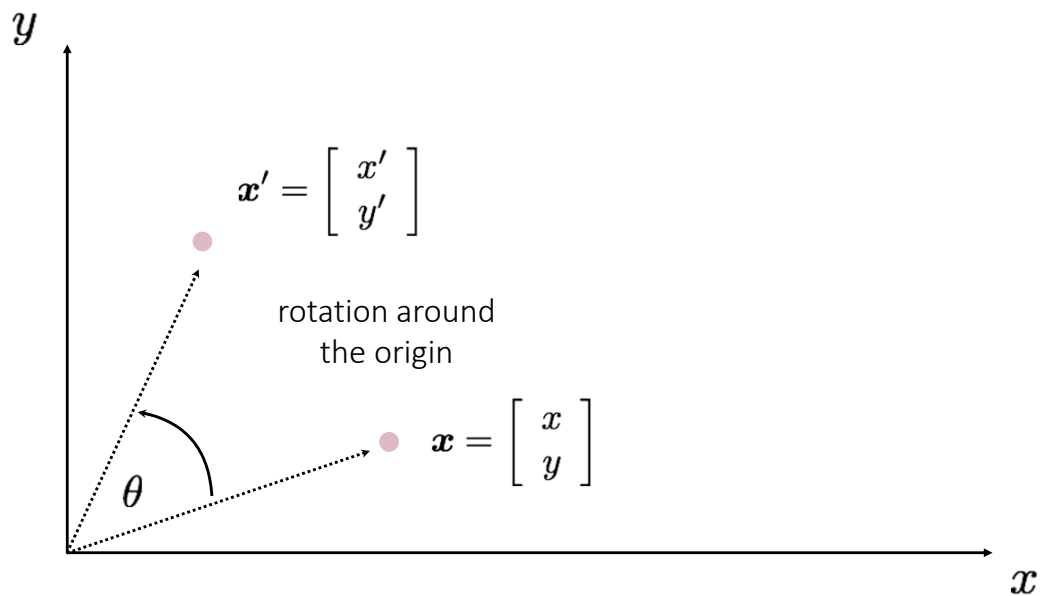
## 2D planar transformations



As in scaling, we can write the shear transform in matrix form, as the matrix shown here: The diagonal is ones, and the anti-diagonal are the shear factors.

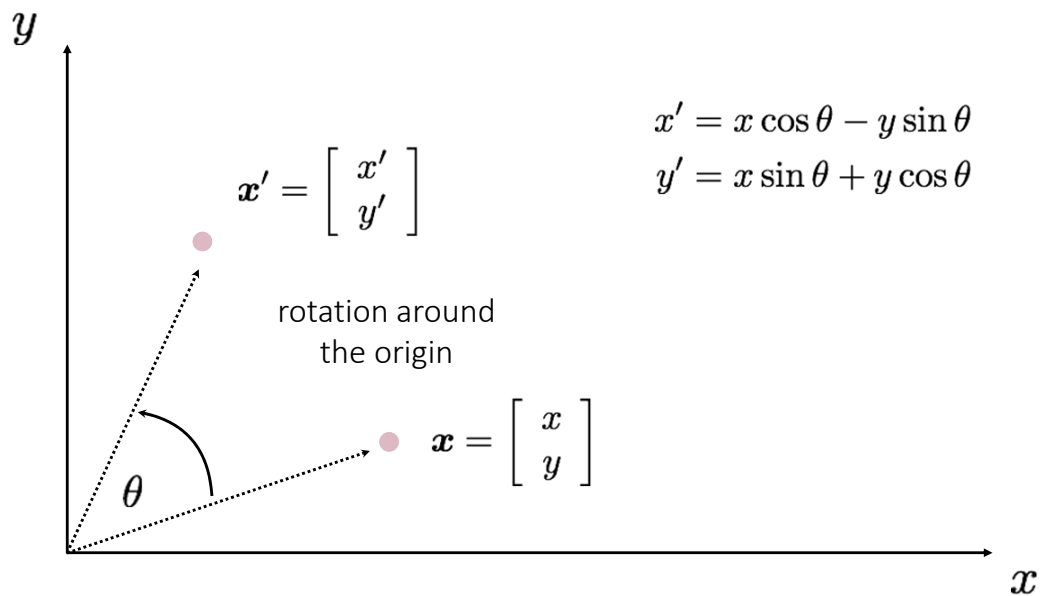


## 2D planar transformations



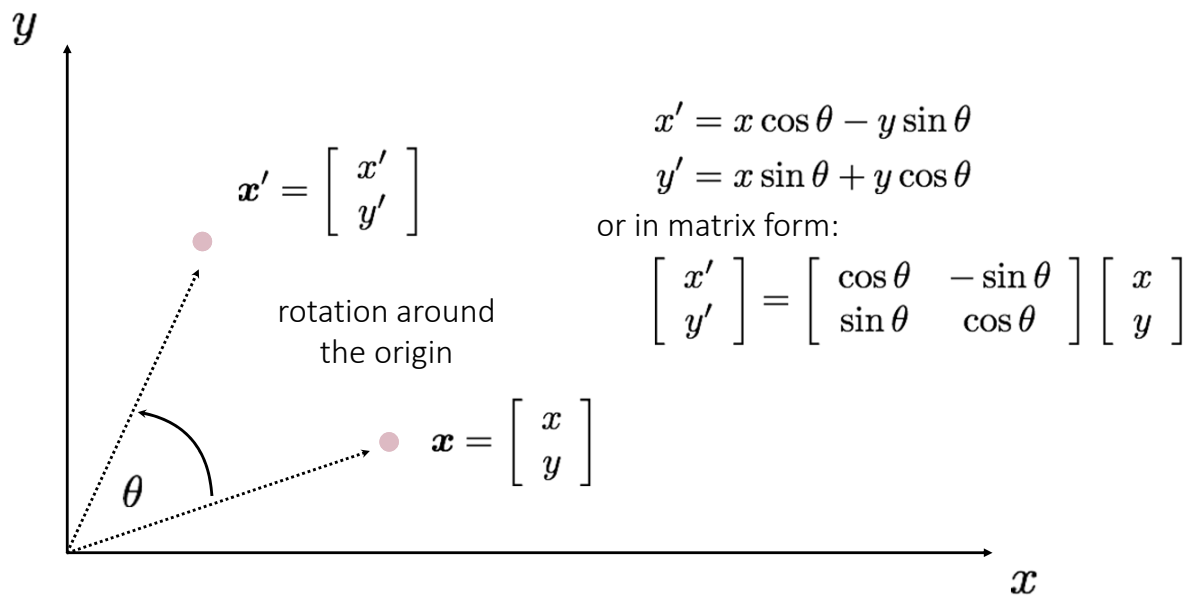
Another very common transformation is rotation *around the origin*. For every 2D point  $x$ - $y$ , we can form the vector connecting it to the origin, and then rotate this vector by some angle  $\theta$ .

## 2D planar transformations



... we end up with this convenient set of equations, involving  $x$ ,  $y$ , and the sine and cosine of  $\theta$ .

## 2D planar transformations



Once again, we can write rotation in matrix form using the matrix shown here, which is completely determined by the rotation angle  $\theta$ .

## 2D planar and linear transformations

$$\mathbf{p}' = f(p; m)$$



$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \mathbf{M} \begin{bmatrix} x \\ y \end{bmatrix}$$

parameters  $m$

point  $p$

To briefly summarize what we have seen so far, we have considered transformation functions that map some point  $x$ - $y$  to another point, with the mapping determined by a set of parameters. By restricting our attention to *linear* 2D transforms, we have seen that they can be expressed as a 2x2 matrix, depending on the same set of parameters, that transforms an  $x$ - $y$  vector through a matrix-vector multiplication.

## 2D planar and linear transformations

Scale

$$\mathbf{M} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

Flip across y

$$\mathbf{M} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$$

Rotate

$$\mathbf{M} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

Flip across origin

$$\mathbf{M} = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}$$

Shear

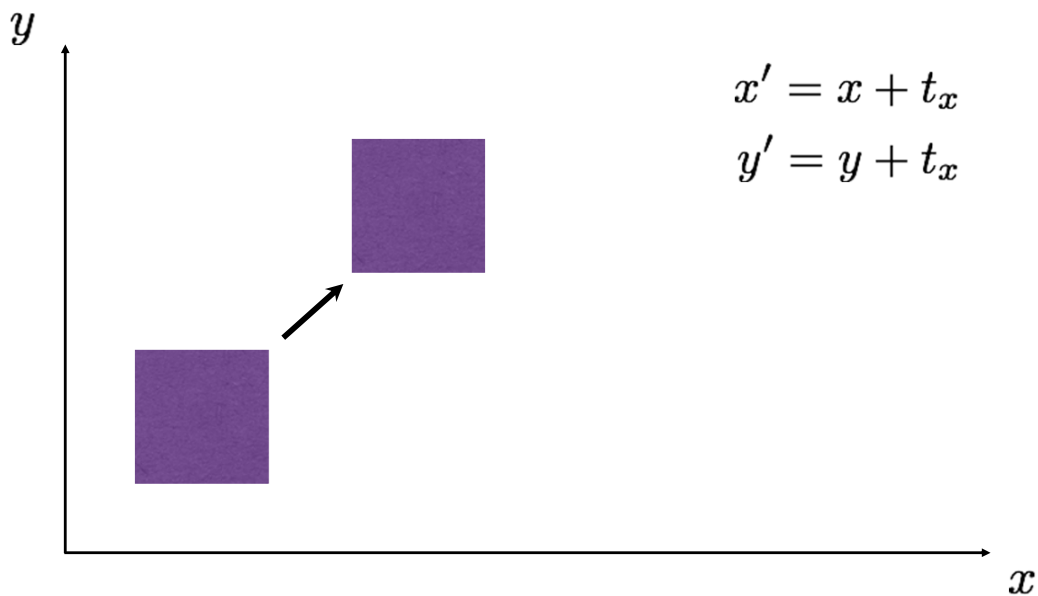
$$\mathbf{M} = \begin{bmatrix} 1 & s_x \\ s_y & 1 \end{bmatrix}$$

Identity

$$\mathbf{M} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

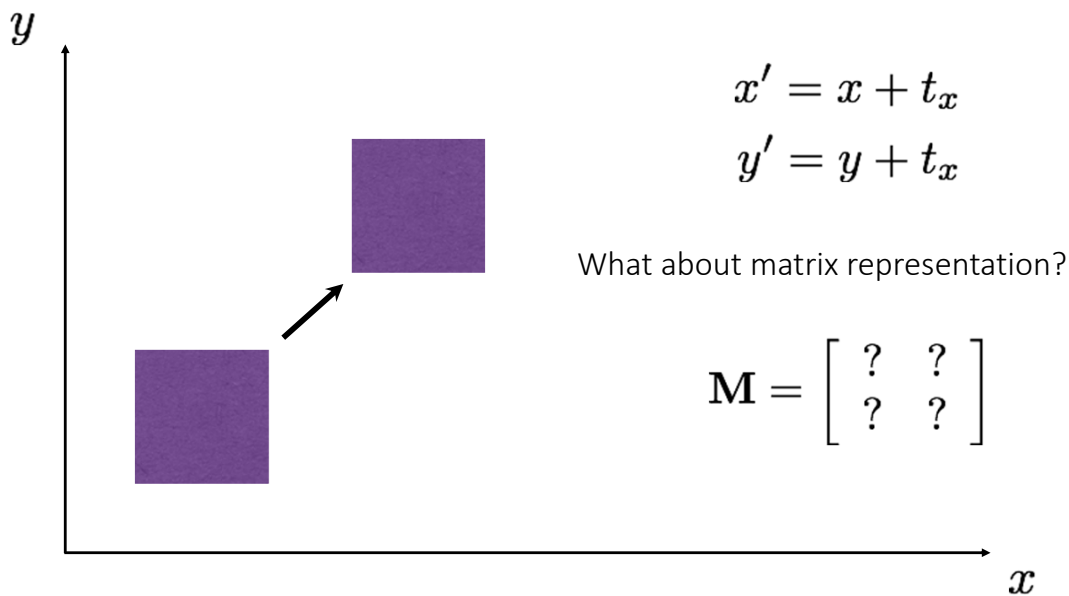
And here is a quick summary of all the 2x2 transformation matrices we have seen so far, as well as some simple extra ones, such as flipping an image across the y axis or across the origin, or just keeping an image the same.

## 2D translation



Now, let's take a look at another 2D transformation, arguably one of the simplest ones: Just translating every pixel by some fixed vector  $t_x$ - $t_y$ . We can very easily express this using two summation equations. Given that this is still a linear transformation, we want to be able to write it as a 2x2 matrix.

## 2D translation



Surprisingly, the answer is that it is impossible to express this very simple transformation as a 2x2 matrix! This is annoying, as expressing transformations in the form of matrix-vector products is very convenient both mathematically and programmatically, making transformations very easy to implement and concatenate with each other. To get a matrix representation for this, and other 2D linear transformations, we will need to change how we represent 2D points x-y.

Homogeneous coordinates



## 2D homogeneous coordinates

heterogeneous      homogeneous  
coordinates      coordinates

$$\begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

← add a 1 here

- Represent 2D point with a 3D vector

We will now introduce a new type of coordinate system that will make working with 2D transformations a lot easier. This new type of coordinates will appear very counter-intuitive at first, but we will be reusing it throughout this and the next lectures. For now, you only need to remember a few simple terms and rules. We will call the x-y vectors we have been working with so far to represent 2D points as the *heterogeneous* coordinates. We will be changing these to homogeneous vectors. Given a heterogeneous coordinate vector, changing it to a homogeneous coordinate vector involves simply adding a third entry equal to 1. So, in homogeneous coordinates, we represent 2D points with a 3D vector.

## 2D homogeneous coordinates

heterogeneous coordinates      homogeneous coordinates

$$\begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \stackrel{\text{def}}{=} \begin{bmatrix} ax \\ ay \\ a \end{bmatrix}$$

- Represent 2D point with a 3D vector
- 3D vectors are only defined up to scale

A very important property of these homogeneous coordinates is that they are defined up to scale: If we take the 3x1 homogeneous coordinate vector, and multiple all of its entries with a scalar  $a$ , the resulting 3x1 homogeneous coordinates will still correspond to the same 2D point.

## 2D homogeneous coordinates

Conversion:

- heterogeneous  $\rightarrow$  homogeneous

$$\begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Scale invariance:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = a \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

So far, we have seen two things: First, a definition of homogeneous coordinates in terms of how we construct them from heterogeneous coordinates. And second, a property of homogeneous coordinates, namely their scale invariance.

## 2D homogeneous coordinates

Conversion:

- heterogeneous  $\rightarrow$  homogeneous

$$\begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- homogeneous  $\rightarrow$  heterogeneous

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} \Rightarrow \begin{bmatrix} x/z \\ y/z \end{bmatrix}$$

Scale invariance:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = a \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

We can use these to come up with a rule for converting from homogeneous coordinates back to heterogeneous ones. All we have to do is form a 2x1 vector where we divide the x and y entries with the z one.

## 2D homogeneous coordinates

Conversion:

- heterogeneous  $\rightarrow$  homogeneous

$$\begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- homogeneous  $\rightarrow$  heterogeneous

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} \Rightarrow \begin{bmatrix} x/z \\ y/z \end{bmatrix}$$

Scale invariance:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = a \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

Special points:

- point at infinity

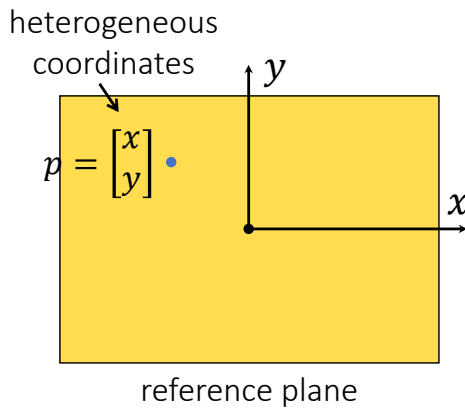
$$\begin{bmatrix} x \\ y \\ 0 \end{bmatrix}$$

- undefined

$$\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

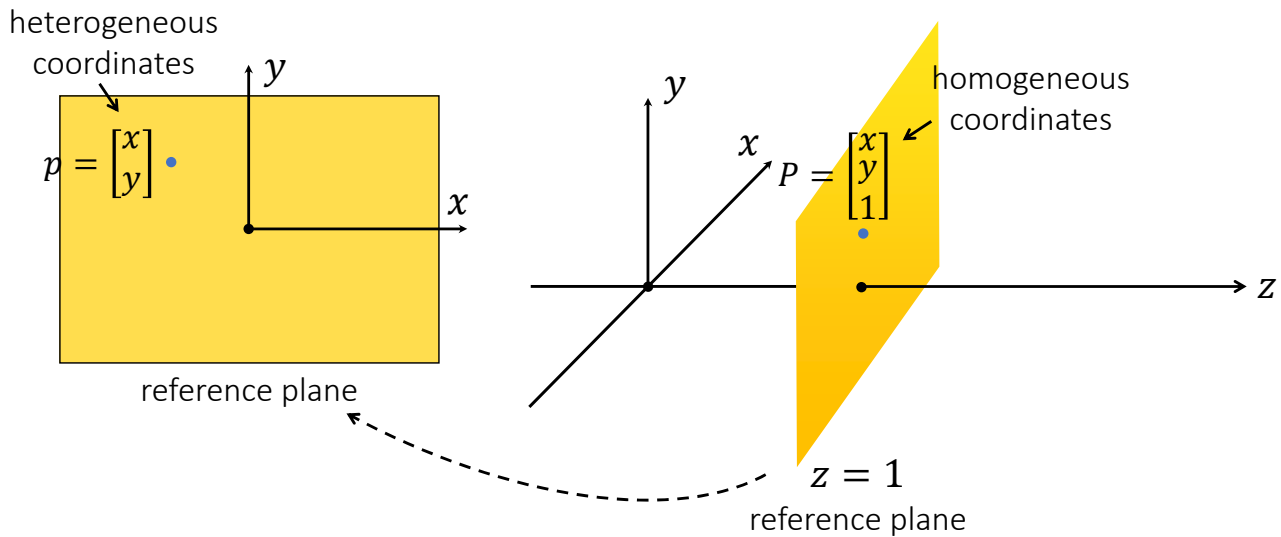
This gives rise to two special cases, when the third entry in the homogeneous coordinate vector is zero. If at least one of the other two coordinates is non-zero, then the homogeneous coordinate vector is representing what we call a point at infinity. We will explain what this means in a later lecture. If all entries are zero, then we end up with an undefined point, in the sense that we cannot map it to heterogeneous coordinates, and thus a 2D point, in a meaningful way.

## 2D projective geometry



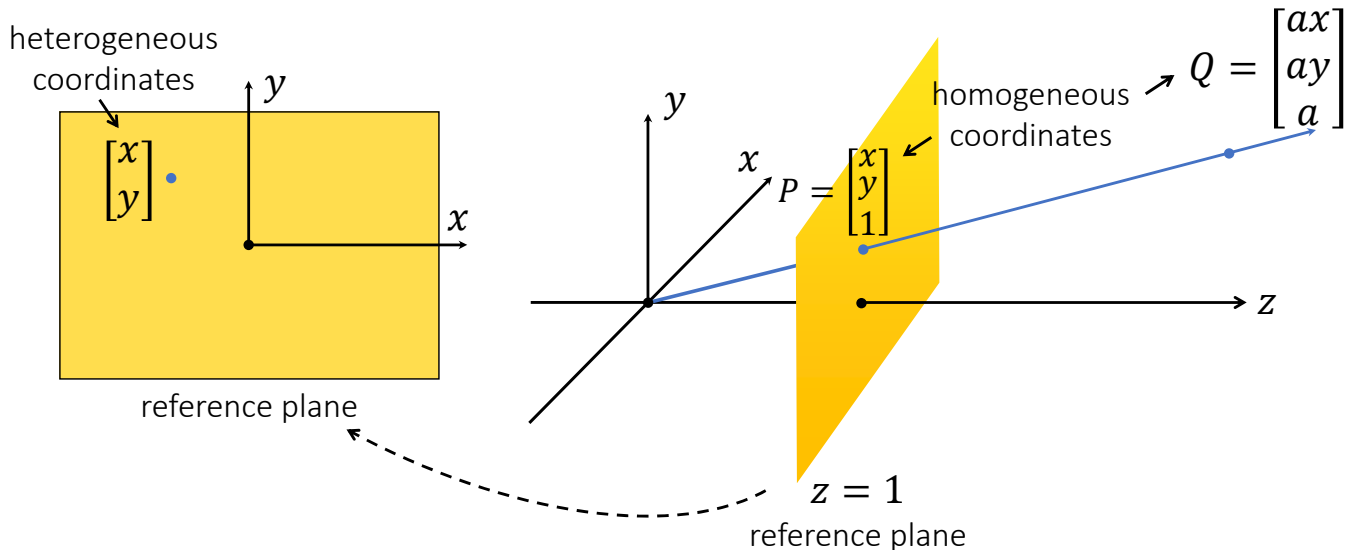
Homogeneous coordinates have been developed in the context of a mathematical area called projective geometry. At a very, very high level, projective geometry gives us a different way to study points on the 2D plane--- the points that we would normally express with the 2x1 heterogeneous coordinates  $x$ - $y$ .

## 2D projective geometry



In particular, projective geometry takes this plane, and embeds it in 3D, as a plane orthogonal to the  $z$  axis, at  $z = 1$ . So, effectively, every  $2 \times 1$  vector of heterogeneous coordinates  $x$ - $y$  in the original plane becomes now a  $3 \times 1$  vector of homogeneous coordinates  $x$ - $y$ -1 on the plane embedded in 3D space.

## 2D projective geometry



Through the scale invariance property, homogeneous coordinates map all points on a line passing through the origin to the point where this line intersects the reference plane.

Now, projective geometry takes this a step further: We can take every point on the 3D plane, and form the line that connects it to the origin. This line will include points of the form  $ax-ay-a$ ; that is, scalar multiples of the homogeneous coordinate vector. Projective geometry says that we will treat all points along this line as if they were the same point. Put another way, given any point in the 3D space, we can map it to a point in the original 2D plane by first forming the line connecting the 3D point to the origin, then finding where this line intersects the  $z = 1$  plane. The diagram we are seeing here will appear again in the Camera lectures, as it is the diagram we will use to describe the operation of cameras.



## 3D homogeneous coordinates

heterogeneous  
coordinates

homogeneous  
coordinates

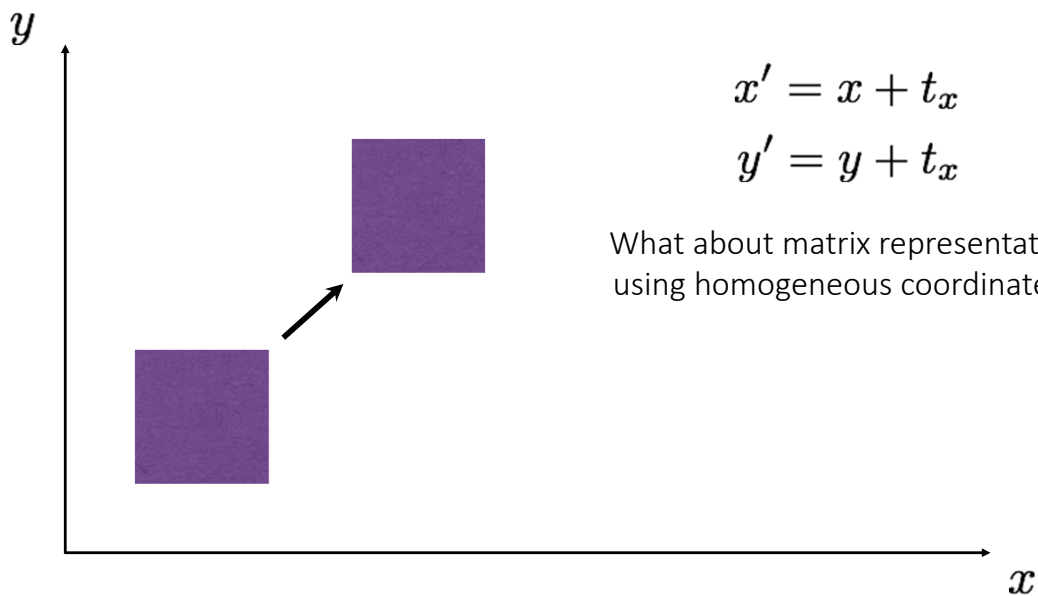
$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \Rightarrow \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \stackrel{\text{def}}{=} \begin{bmatrix} aX \\ aY \\ aZ \\ a \end{bmatrix}$$

- Represent 3D point with a 4D vector
- 4D vectors are only defined up to scale

A last thing I want to mention before we go back to transformations is that we can define homogeneous coordinates for 3D points as well. In this case, the heterogeneous coordinates are a 3x1 vector, and the homogeneous coordinates are the 4x1 vector. All the conversion and invariance properties we discussed for 2D homogeneous coordinates are also true for 3D homogeneous coordinates. We won't be needing 3D homogeneous coordinates in this lecture, but we will be using them in the next lectures.

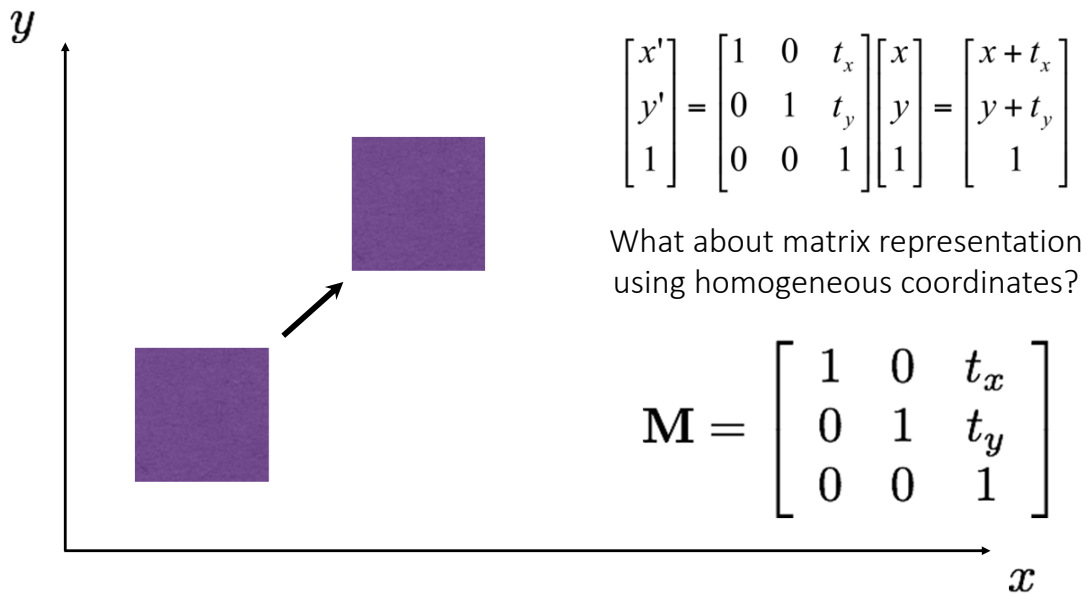
2D transformations in homogeneous coordinates

## 2D translation



Let's now go back to transformations, and see why we went into the trouble of introducing these strange homogeneous coordinates. We left off at translation, where we were stuck about how to express translation in matrix form. As I mentioned, it is impossible to derive a matrix expression using heterogeneous coordinates. But can we do so using homogeneous coordinates?

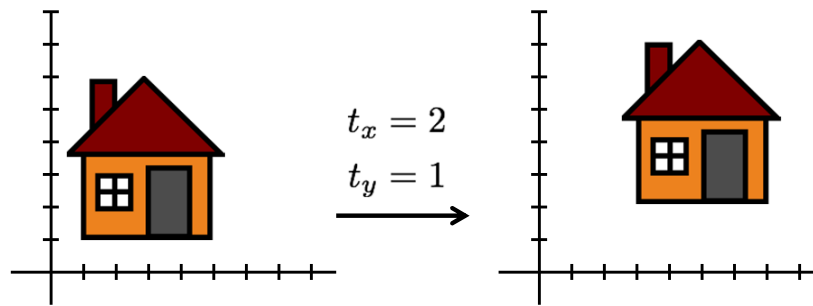
## 2D translation in homogeneous coordinates



The answer is yes! The transformation matrix will now be 3x3, and not 2x2 as it were before, given that we are transforming 3x1 homogeneous coordinate vectors. The special extra dimension we added to go from heterogeneous to homogeneous coordinates now makes it possible to express translation in matrix form.

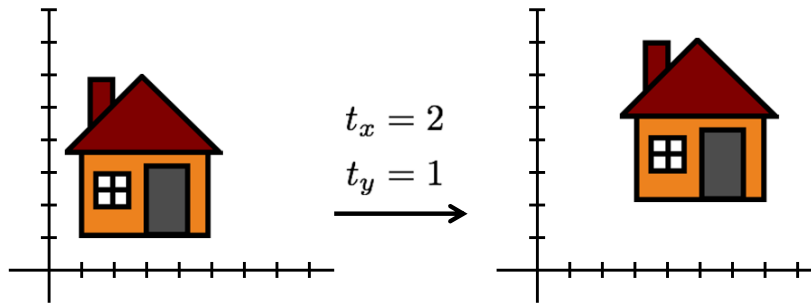
## 2D translation using homogeneous coordinates

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix}$$



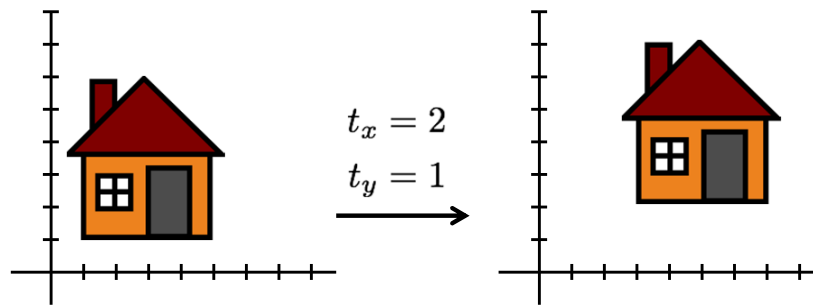
## 2D translation using homogeneous coordinates

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} ax \\ ay \\ a1 \end{bmatrix} = ?$$



## 2D translation using homogeneous coordinates

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} ax \\ ay \\ a1 \end{bmatrix} = \begin{bmatrix} a(x + t_x) \\ a(y + t_y) \\ a \end{bmatrix} = \begin{bmatrix} (x + t_x) \\ (y + t_y) \\ 1 \end{bmatrix}$$



## 2D transformations in homogeneous coordinates

Re-write these transformations as 3x3 matrices:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

translation

scaling

rotation

shearing

We can revisit all of the transformations we saw earlier, and express them in homogeneous coordinates as well.



## 2D transformations in homogeneous coordinates

Re-write these transformations as 3x3 matrices:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

translation

rotation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

scaling

shearing

For example, here is the 3x3 transformation matrix for scaling.

## 2D transformations in homogeneous coordinates

Re-write these transformations as 3x3 matrices:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

translation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

scaling

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & \beta_x & 0 \\ \beta_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

rotation

shearing

## 2D transformations in homogeneous coordinates

Re-write these transformations as 3x3 matrices:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

translation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

scaling

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

rotation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & \beta_x & 0 \\ \beta_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

shearing

## Matrix composition

Transformations can be combined by matrix multiplication:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \left( \begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \right) \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

$\mathbf{p}'$     =    translation( $t_x, t_y$ )                  rotation( $\theta$ )                  scale( $s, s$ )                   $\mathbf{p}$

Before we continue, it is worth taking a moment here to see one of the reasons why we insist on expressing 2D transformations as matrices. Let's say that we want to apply multiple transformations, for example a scaling, then a rotation, then a translation. We can do so by multiplying the transformation matrices for each of these transformations.

## Matrix composition

Transformations can be combined by matrix multiplication:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \left( \begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \right) \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

$\mathbf{p}' = \text{translation}(t_x, t_y) \quad \text{rotation}(\theta) \quad \text{scale}(s, s) \quad \mathbf{p}$

Multiplication order matters! Transformations are applied from right to left.

Matrix multiplication is not commutative. And the multiplication order matters: transformations are applied, in order from right to left. For example, here we first apply scaling.

## Matrix composition

Transformations can be combined by matrix multiplication:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \left( \begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \right) \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

$\mathbf{p}' = \text{translation}(t_x, t_y) \text{ rotation}(\theta) \text{ scale}(s, s) \mathbf{p}$

Multiplication order matters! Transformations are applied from right to left.

Then rotation

## Matrix composition

Transformations can be combined by matrix multiplication:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \left( \begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \right) \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

$$\mathbf{p}' = \text{translation}(t_x, t_y) \quad \text{rotation}(\theta) \quad \text{scale}(s, s) \quad \mathbf{p}$$

Multiplication order matters! Transformations are applied from right to left.

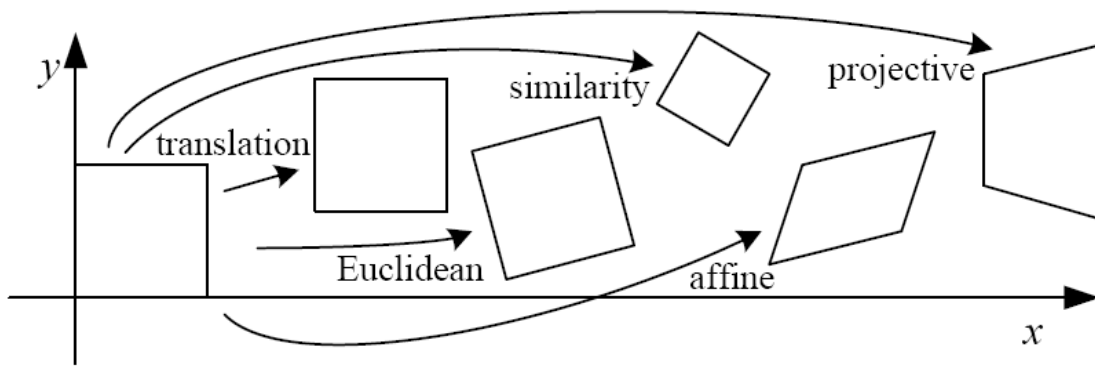
And lastly translation.

## Classification of 2D transformations

Expressing 2D transformations as matrices gives us other advantages. For example, it makes it easy to categorize different types of transformations, which is what we will cover next. It also let's us derive efficient algorithms for determining unknown transformations, as we will see later in this lecture.



## Classification of 2D transformations



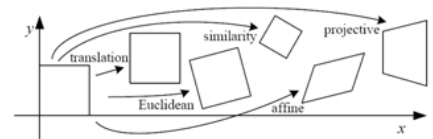
We can now use the matrix representation in homogeneous coordinates of 2D transformations, to more systematically categorize them into different, progressively more general groups. Each group we will see will subsume all previous ones.

# Classification of 2D transformations

Translation:

$$\begin{bmatrix} 1 & 0 & t_1 \\ 0 & 1 & t_2 \\ 0 & 0 & 1 \end{bmatrix}$$

degrees of freedom? 2



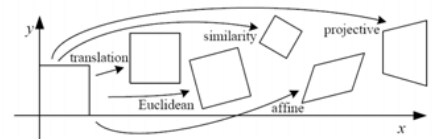
The first, simplest category will be the translation transformations we have already seen, which correspond to matrices of the form shown here. As we categorize transformations, it will be helpful to think about the degrees of freedom we have available in specifying each transformation. This is the number of *independent* parameters we need to provide, in order to be able to write out the 3x3 matrix corresponding to each transformation. For translation transformations, the degrees of freedom are equal to 2.

# Classification of 2D transformations

Euclidean (a.k.a. rigid):  
rotation + translation

$$\begin{bmatrix} \cos \theta & -\sin \theta & t_1 \\ \sin \theta & \cos \theta & t_2 \\ 0 & 0 & 1 \end{bmatrix}$$

degrees of freedom: 3



The next more general transformation category will be Euclidean transforms, which combine rotations and translations. These are also called rigid transformations, because they do not deform the object they are applied to: they simply move it around and change its pose. A Euclidean transform has 3 degrees of freedom, two for specifying translation, and one for specifying rotation.

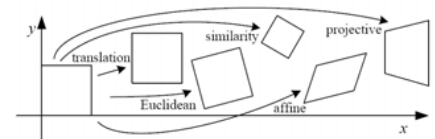
# Classification of 2D transformations

changing this matrix entry affects  
three other matrix entries

Euclidean (a.k.a. rigid):  
rotation + translation

$$\begin{bmatrix} \cos \theta & -\sin \theta & t_1 \\ \sin \theta & \cos \theta & t_2 \\ 0 & 0 & 1 \end{bmatrix}$$

degrees of freedom: 3



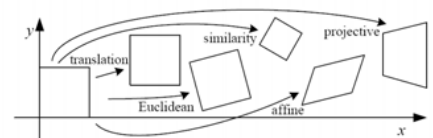
It is worth pausing for a second here, to examine again what we mean by degrees of freedom. Note that this is *not* the same as the number of entries in the 3x3 matrix that can change as we vary the transformation. This is because not all entries may be changing independently of each other. For example, in the case of a Euclidean transformation, changing the top-left entry will also affect the three other matrix entries immediately adjacent to it. The degrees of freedom of a transformation determine how many parameters we get to control *independently*, with each parameter potentially affecting more than one entry in the 3x3 transformation matrix.

# Classification of 2D transformations

Similarity:  
uniform scaling + rotation  
+ translation

$$\begin{bmatrix} s \cdot \cos \theta & -s \cdot \sin \theta & t_1 \\ s \cdot \sin \theta & s \cdot \cos \theta & t_2 \\ 0 & 0 & 1 \end{bmatrix}$$

degrees of freedom: 4



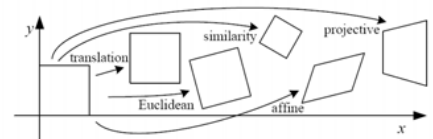
The next transformation category is similarity transformations, where in addition to rotation and translation, we can do *uniform* scaling. The degrees of freedom we have here are 4, 2 for translation, one for rotation, and one for scaling.

# Classification of 2D transformations

**Affine:**  
uniform scaling + shearing  
+ rotation + translation

$$\begin{bmatrix} r_1 & r_2 & r_3 \\ r_4 & r_5 & r_6 \\ 0 & 0 & 1 \end{bmatrix}$$

degrees of freedom: 6



Next up are affine transforms, which combine uniform scaling, shearing, rotation, and translation (at and order). The degrees of freedom of an affine transform are 6. This means that we can independently control each of the six entries at the top 2 rows of the transformation matrix, while leaving the last row fixed to 0-0-1.

# Classification of 2D transformations

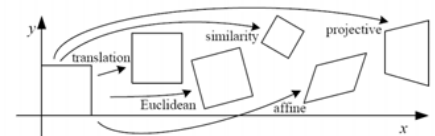
$$\begin{bmatrix} s \cdot \cos \theta & -s \cdot \sin \theta \\ s \cdot \sin \theta & s \cdot \cos \theta \end{bmatrix} \cdot \begin{bmatrix} 1 & h_1 \\ h_2 & 1 \end{bmatrix} = \begin{bmatrix} s \cdot \cos \theta - h_2 \cdot s \cdot \sin \theta & -s \cdot \sin \theta + h_1 \cdot s \cdot \cos \theta \\ s \cdot \sin \theta + h_2 \cdot s \cdot \cos \theta & s \cdot \cos \theta + h_1 \cdot s \cdot \sin \theta \end{bmatrix} \equiv \begin{bmatrix} r_1 & r_2 \\ r_4 & r_5 \end{bmatrix}$$

similarity (2 DOF)
shear (2 DOF)
composition (4 DOF)

**Affine:**  
uniform scaling + shearing  
+ rotation + translation

$$\begin{bmatrix} r_1 & r_2 & r_3 \\ r_4 & r_5 & r_6 \\ 0 & 0 & 1 \end{bmatrix}$$

degrees of freedom: 6



It may not be immediately clear why adding shearing to a similarity transform gives us six degrees of freedom. You may suspect that some of the six matrix entries will be connected to each other. To see why we have six degrees of freedom, let's look at the top-left 2x2 square of the matrix. This is formed as the composition of a similarity and a shear transform, for a total of 4 degrees of freedom. Also, we can show (will not prove) that any 2x2 matrix you can decompose as a product of a similarity matrix and a shearing matrix. Equivalently, this means that we can control each of the entries in this 2x2 sub-matrix independently of each other. Together with two degrees of freedom for translation, we get a total of six.

# Affine transformations

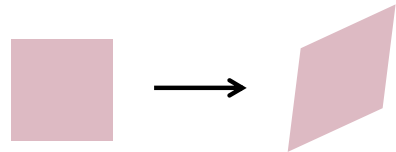
Affine transformations are combinations of

- arbitrary (4 DOF) linear transformations; and
- translations

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Properties of affine transformations:

- origin does not necessarily map to origin
- lines map to lines
- parallel lines map to parallel lines
- Area ratios are preserved
- compositions of affine transforms are also affine transforms
- last coordinate does not change



Affine transformations will be one of the two main types of transformations we will be using in this and the next four lectures. So, it is worth taking a closer look at some of the properties. Affine transforms, being linear transforms, map lines to lines. They also preserve a lot of properties of the image they are transforming, for example parallel lines remain parallel, and ratios of areas are preserved after mapping. Additionally, compositing multiple affine transforms through matrix multiplication produces another affine transform---mathematically, we say that affine transforms are closed under composition.



# Affine transformations

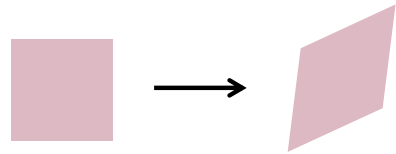
Affine transformations are combinations of

- arbitrary (4 DOF) linear transformations; and
- translations

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Properties of affine transformations:

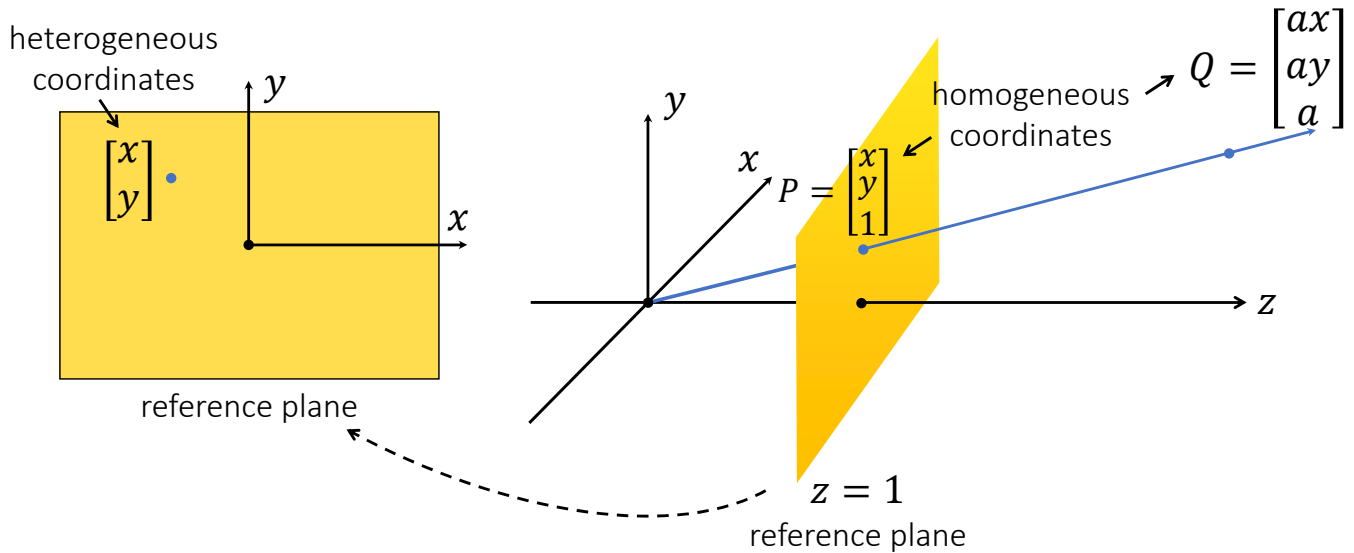
- origin does not necessarily map to origin
- lines map to lines
- parallel lines map to parallel lines
- Area ratios are preserved
- compositions of affine transforms are also affine transforms
- last coordinate does not change



Lastly, because the last row of an affine transform is always 0-0-1, applying an affine transform to a homogeneous coordinate vector leaves the last coordinate unchanged.

This last property can help us get some intuition about what kinds of transforms we can model as affine transforms.

## How to interpret affine transformations here?

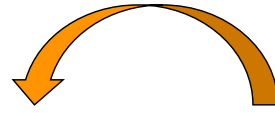


By keeping the last coordinate constant, an affine transformation maps points on a  $z = c$  plane to points on the same plane.

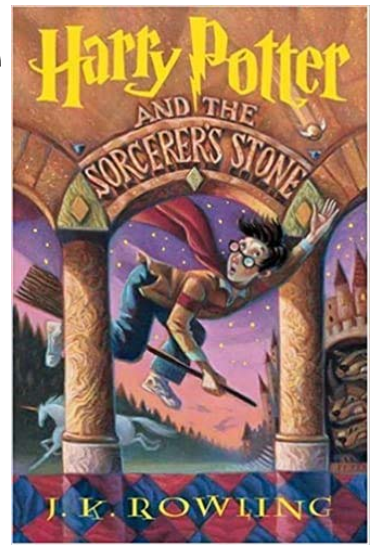
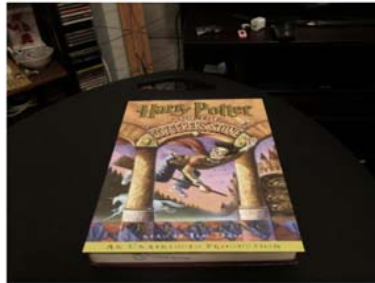
For that, let's go back to the projective geometry view of homogeneous coordinates. Remember, in homogeneous coordinates, we map the 2D space to the  $z = 1$  plane embedded in 3D space.

In this world view, affine transforms are the ones that map 3D homogeneous coordinate vectors in such a way that they stay in the same  $z$  plane.

## Beyond affine transformations



This is not an affine transformation



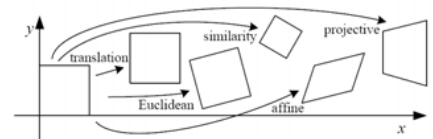
e.g. parallel lines do not map to parallel lines

# Classification of 2D transformations

Projective (a.k.a. homography):

$$\begin{bmatrix} r_1 & r_2 & r_3 \\ r_4 & r_5 & r_6 \\ r_7 & r_8 & r_9 \end{bmatrix}$$

degrees of freedom: **8** (homogeneous coordinates, and therefore transformation matrices, are defined up to scale)



Now, let's look at the last general type of transformation we will consider. We will be calling this by two names, a projective transformation, or a homography. In this case, we get to change all nine entries of the 3x3 transformation matrix. This gives us 8 degrees of freedom, despite the fact that we are changing nine entries. The reason is that, as homogeneous coordinates are defined up to scale, so are transformations applied to homogeneous coordinates. Therefore, all 3x3 matrices that are scalar multiples of each other correspond to the same transformation. This reduces our degrees of freedom to 8.

# Projective transformations (a.k.a homographies)

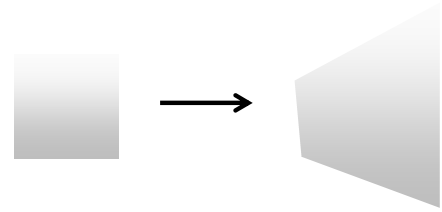
Projective transformations are combinations of

- affine transformations; and
- projective warps

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

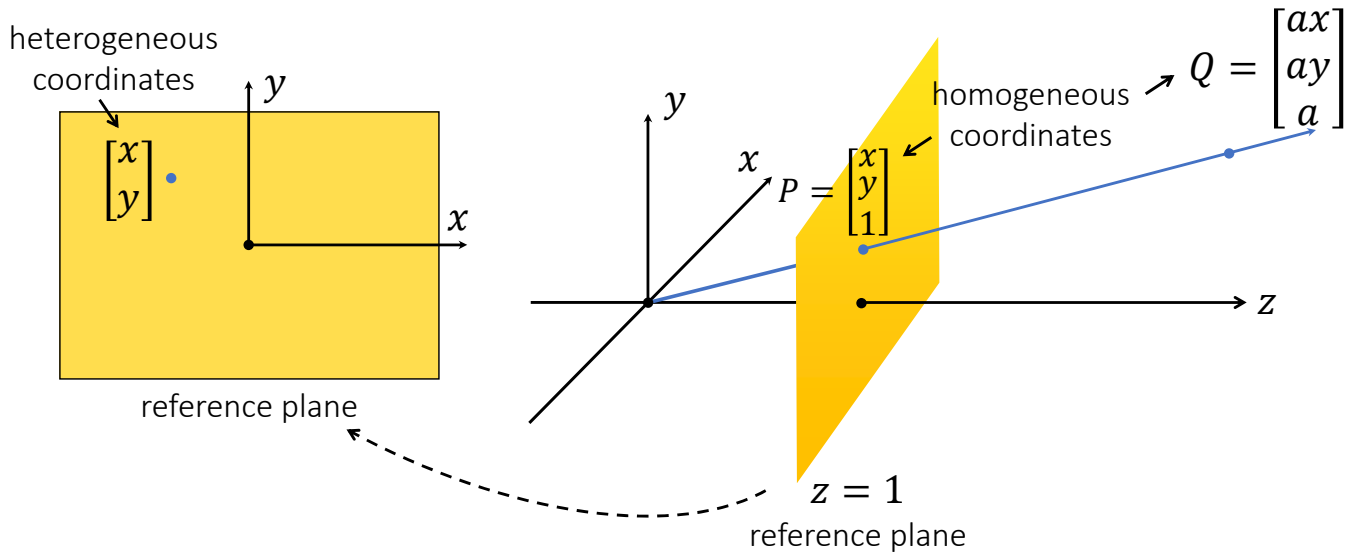
Properties of projective transformations:

- origin does not necessarily map to origin
- lines map to lines
- parallel lines do not necessarily map to parallel lines
- ratios are not necessarily preserved
- compositions of projective transforms are also projective transforms



Projective transformations break some of the properties we had for affine transforms. For example, parallel lines are not necessarily parallel after transformation, and area and length ratios are not necessarily preserved.

## How to interpret projective transformations here?



By changing the last coordinate, a projective transformation does *not* map points on a  $z = c$  plane to points on the same plane.

Going back to our projective geometry diagram, projective transformations can map a point from one  $z$  plane to another, unlike affine transforms.

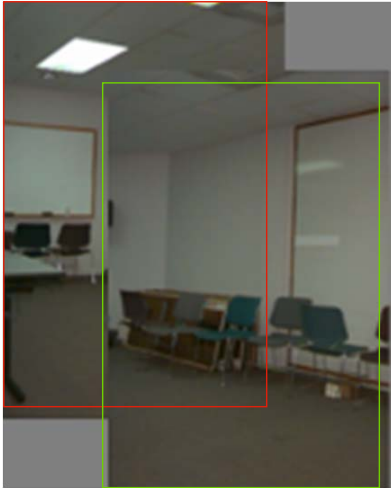
## Classification of 2D transformations

| Name              | Matrix                                                                                                                                           | # D.O.F. |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| translation       | $\begin{bmatrix} 1 & 0 & t_1 \\ 0 & 1 & t_2 \\ 0 & 0 & 1 \end{bmatrix}$                                                                          | 2        |
| rigid (Euclidean) | $\begin{bmatrix} \cos \theta & -\sin \theta & t_1 \\ \sin \theta & \cos \theta & t_2 \\ 0 & 0 & 1 \end{bmatrix}$                                 | 3        |
| similarity        | $\begin{bmatrix} s \cdot \cos \theta & -s \cdot \sin \theta & t_1 \\ s \cdot \sin \theta & s \cdot \cos \theta & t_2 \\ 0 & 0 & 1 \end{bmatrix}$ | 4        |
| affine            | $\begin{bmatrix} r_1 & r_2 & r_3 \\ r_4 & r_5 & r_6 \\ 0 & 0 & 1 \end{bmatrix}$                                                                  | 6        |
| projective        | $\begin{bmatrix} r_1 & r_2 & r_3 \\ r_4 & r_5 & r_6 \\ r_7 & r_8 & r_9 \end{bmatrix}$                                                            | 8        |

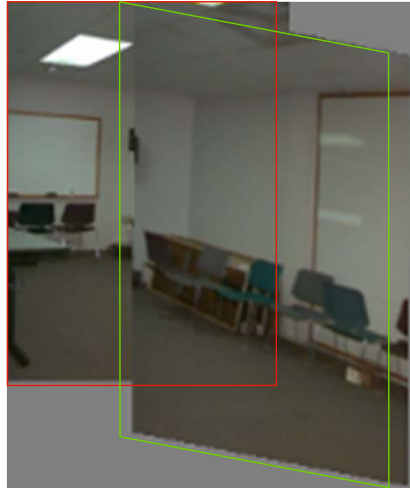
So, to conclude this part of the lecture, this is a summary of all of the different types of transformations we have seen, along with their corresponding transformation matrices and degrees of freedom.

# Warping with different transformations

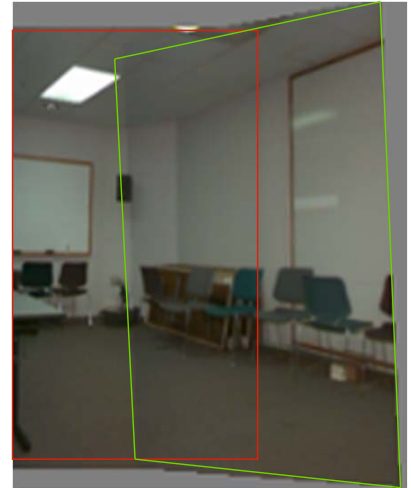
translation



affine



Projective (homography)





## Applications of image homographies

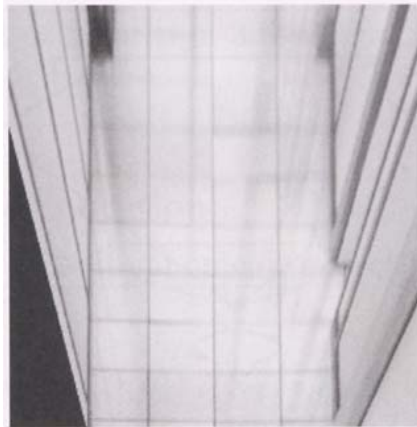
we will take a closer look at the types of applications that can be enabled, by considering the most general of the 2D linear transformations we saw, homographies. As we mentioned earlier, unlike affine transforms, homographies allow us to map points from one plane to another. Effectively, they make it possible to change the perspective from which we are observing a plane.

## View warping

original view



synthetic top view



synthetic side view



Let's look at an example of this. Consider this corridor scene, which we are observing from a particular view. In a VR application, we may want to be able to virtually “navigate” this scene, by observing it from different views.

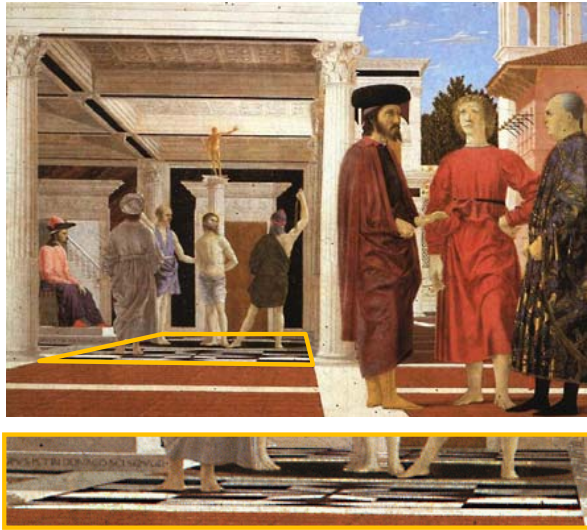
If we know that the floor or the walls on the scene are planar, then without any other information about the scene except for the original image, we can use homographies to synthesize new views of it, for example from the top or the side.

## Understanding geometric patterns



Here is a financier example of the same thing. Consider this painting from the renaissance era.

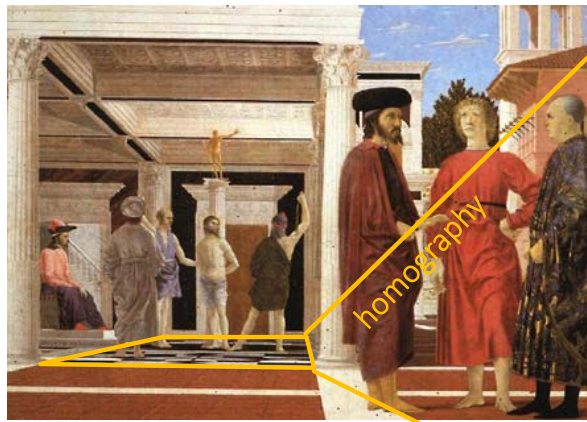
## Understanding geometric patterns



magnified view of floor

If we look closely, in the scene, we show that the painting includes an elaborate tile pattern for the floor that the figures to the left are standing on.

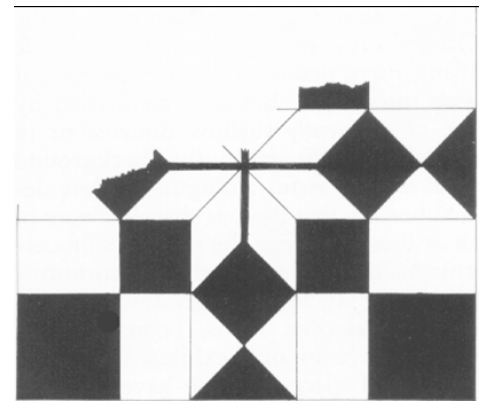
## Understanding geometric patterns



magnified view of floor



rectified view

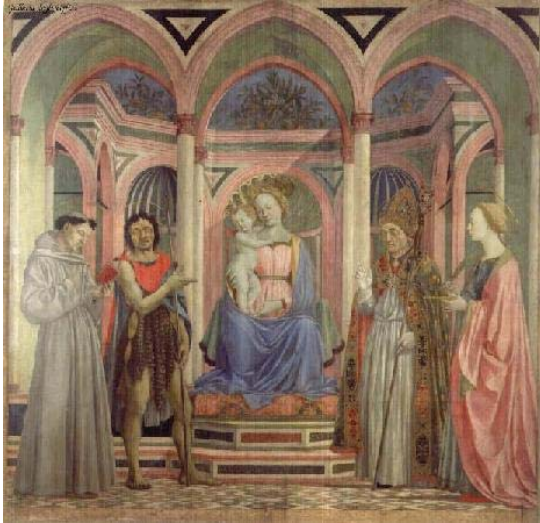


reconstruction from  
rectified view

Without any other information about the scene, we can use a homography to synthesize a view of this floor pattern from the top, and reconstruct it in detail.

# Understanding geometric patterns

Very popular in renaissance drawings (when perspective was discovered)



rectified view  
of floor



reconstruction

Here is another example of the same. This type of hidden geometric details were very popular in renaissance drawings, when the rules of perspective and homographies were discovered.

## Image rectification

two  
original  
images



rectified and stitched

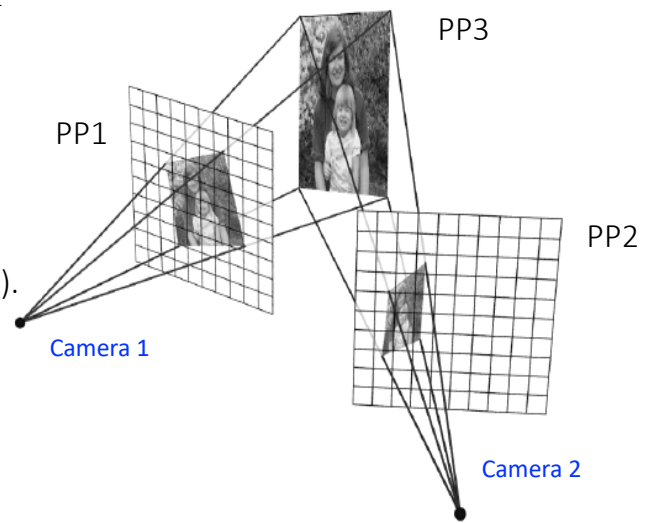
Here is yet another example. We are given two photographs of the facades of two adjacent buildings. We know that, from construction, these facades should be rectangular and planar.

We can then compute a homography to transform these two images into rectified views of the buildings, where the facades are indeed rectangular, as well as stitch them to one photograph.

# Mapping with homographies

Which kind of transformation is needed to warp projective plane 1 into projective plane 2?

- A projective transformation (a.k.a. a homography).



Theorem: Every image of a *plane* in camera 1 can be mapped into an image of a plane in camera 2 via a 3x3 projective homography

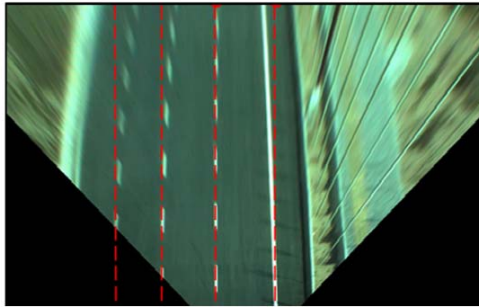
$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

In general if we observe an image of a plane from one viewpoint and an image of the plane from another viewpoint we can use an homography to map between them.  
This is a theorem which we will prove in a later lecture.





Original Road Image



Homography maps the plane correctly, but mess other objects (e.g. trees not on the plane)

Note an homography can only map a plane.  
If the scene contains other objects not on the same plane they will not be mapped correctly.

## Creating panoramas

1. Capture multiple images from different viewpoints by rotating a camera.



2. Use homographies to stitch them together into a virtual wide-angle image.



here is an example that was one of the first widely commercialized applications of computer vision. This is a capability that you can now find on most cell phone cameras: Let's say that we want to create a very wide-angle panorama of some scene, but we do not have access to the expensive wide-angle lens required to capture the scene. What we can do is first take multiple narrow-angle images of the scene by simply rotating our camera around a fixed point. Then, we can use homographies to warp and stitch all of the images together, into a single virtual wide-angle panoramic image.

## We can use homographies when...

1. ... the scene is planar; or



2. ... the scene is very far or has small (relative) depth variation  
→ scene is approximately planar



To summarize all of these: We can use homographies any time we are dealing with scenes that are planar. We can also use homographies on scenes that are so far away or have so small relative depth variations, that they are *approximately* planar.

## We can use homographies when...

3. ... the scene is captured under camera rotation only (no translation or pose change)



Lastly, we can use homographies on photographs of non-planar scenes, captured under a camera that *only* rotates, and does not otherwise translate or change pose. This last case is probably less intuitive than the other two. We will come back to it and prove it later in the class.

# What's ahead

- Determining unknown affine transformations
- The direct linear transform
- Random sample consensus
- Forward and reverse warping

103

In the rest of the lecture, we will cover a few topics, such as how to robustly estimate unknown affine and homography transformations, and how to implement warping in practice.

## Solving linear systems

Next we want to talk on how to compute homographies.  
before we see all that, we will need to make a short detour, to review a few important topics from linear algebra.

## Heterogeneous linear systems

$$A \cdot x = b$$

we will be reusing the ideas of the next slides several times throughout this and the next 4 lectures. So it is important to understand and remember what we cover here. We will start by seeing how we can solve *heterogeneous* linear systems.

## Heterogeneous linear systems

$$A \cdot x = b$$

heterogeneous: right hand side **b** is non-zero

By heterogeneous, we mean linear systems where the right hand side is non-zero.



## Heterogeneous linear systems

$$A \cdot x = b$$

dimensions:     $M \times N$      $N \times 1$      $M \times 1$

The number of solutions to the system will depend on the dimensions of the quantities that show up in it. In particular, we have a matrix  $A$  of size  $M \times N$ , a vector of unknowns  $x$  of size  $N \times 1$ , and the right-hand-side vector  $b$  of size  $M \times 1$ .

## Heterogeneous linear systems

$$A \cdot x = b$$

dimensions:  $M \times N$     $N \times 1$     $M \times 1$

Three cases.

- $M < N$ : the system has infinite solutions (**underdetermined system**)
  - $M = N$ : the system has a unique solution  $A^{-1}b$  if  $A$  is invertible, otherwise it has infinite solutions
  - $M > N$ : the system *typically* has no solutions (**overdetermined system**)
- We are typically interested in the  $M \gg N$  case

We can then separate between three cases: If  $M$  is smaller than  $N$ , we say that our system is underdetermined---we have fewer constraints than unknowns. In this case, our system has infinite solutions. If  $M$  equals  $N$ , and the matrix  $A$  is invertible, then we have exactly as many constraints as unknowns and the system has a unique solution, which can be expressed using the inverse of  $A$ . If  $A$  is not invertible, then the system again has infinite solutions. Lastly, if  $M$  is greater than  $N$ , we say that our system is overdetermined, and we have more constraints than unknowns. The system *typically* has no solutions, unless many of these constraints are linearly dependent.

In both this and the next four lectures, we will mostly be focusing on the overdetermined case, and especially in the case where  $M$  is way larger than  $N$ , so that we are near-guaranteed to have more than  $N$  independent constraints on our unknowns.

## Least-squares solution to heterogeneous linear systems

Replace the linear system:

$$\begin{matrix} & A & \cdot & x & = & b \\ & M \times N & & N \times 1 & & M \times 1 \end{matrix}$$

with the optimization problem:

$$\min_x \|A \cdot x - b\|^2$$

- $M = N$ : same solution as original system
- $M > N$ : find  $x$  that comes *closest* to being a solution in the least-squares sense

We will look at how to obtain so-called least-squares solutions to heterogeneous linear systems. By this we mean that we will be converting the linear system to an optimization problem such as this.

When  $M$  and  $N$  are equal, the solution to the optimization problem will be the same as the solution of the original system. When  $M$  is greater than  $N$ , and the original overdetermined system does not have an exact solution, the optimization problem will let us determine the  $x$  that comes *closest* to being a solution, in the least squares sense.

## Solving the linear least squares optimization

Take the loss function:

$$E_{\text{LLS}} = \|A \cdot x - b\|^2$$

Expand the loss function:

$$E_{\text{LLS}} = x^T (A^T A) x - 2x^T (A^T b) + \|b\|^2$$

Compute the derivative of the loss function:

$$\frac{dE_{\text{LLS}}}{dx} = 2(A^T A)x - 2(A^T b)$$

Compute the derivative of the loss function:

$$\frac{dE_{\text{LLS}}}{dx} = 0 \Rightarrow 2(A^T A)x - 2(A^T b) = 0 \Rightarrow (A^T A)x = A^T b$$

Solve the resulting heterogeneous linear system:

$$x = (A^T A)^{-1} A^T b$$

Pseudo-inverse of  $M \times N$  matrix  $A$ :

$$(A^T A)^{-1} A^T$$

- $M = N$ : equal to  $A^{-1}$ , same solution as original system
- $M > N$ :  $A^T A$  square  $N \times N$  matrix, typically invertible when  $M \gg N$ .

Let's now see how we can solve this least-squares optimization. We can take the loss function of our optimization.

And use the properties of the L2 vector norm to expand it out.

We then compute the derivative of the loss function with respect to our unknowns  $x$ .

The solution to our optimization problem will be at the place where the derivative equals zero. Setting the derivative to zero and rearranging terms gives us a new heterogeneous linear system, that now involves the matrix  $A$ -transpose- $A$  instead of the original matrix  $A$ . This matrix is a square matrix of size  $N \times N$ , which matches the size  $N \times 1$  of the unknown vector  $x$ .

Lastly, we can now solve this heterogeneous linear system, to obtain the solution to our least-squares optimization problem.

It is worth taking a look at the matrix that shows up in this solution. We call this the pseudoinverse of the original  $M \times N$  matrix  $A$ . When  $M$  and  $N$  are equal to

each other, then the pseudo-inverse is the same as the inverse of  $A$ , and therefore the solution to the least-squares optimization problem is the same as the solution to the original system. When  $M$  is a lot greater than  $N$ , the matrix  $A^T A$  will typically be invertible, and therefore we can compute the pseudoinverse and get the least-squares solution to the heterogeneous system.

## Heterogeneous linear systems

$$A \cdot x = b$$

dimensions:  $M \times N$     $N \times 1$     $M \times 1$

So far, we have seen how to solve heterogeneous linear systems.

We will now look at how to solve homogeneous linear systems.

## Homogeneous linear systems

$$A \cdot x = 0$$

homogeneous: right hand side is zero

dimensions:  $M \times N$     $N \times 1$     $M \times 1$

Three cases.

- $M < N$ : the system has at least one solution, the trivial solution  $x = 0$
  - $M = N$ : the system has at least one solution, the trivial solution  $x = 0$
  - $M > N$ : the system has at least one solution, the trivial solution  $x = 0$
- We want to avoid the trivial solution

The difference is that, unlike before, the right hand size will be zero.

This seemingly small change has a pretty significant effect. Namely, no matter whether the system is under or over-determined, it will always have the trivial solution  $x = 0$ .

Typically, this is an uninformative solution that we will be looking to avoid.

## Least-squares solution to homogeneous linear systems

Replace the linear system:

$$\underset{M \times N}{A} \cdot \underset{N \times 1}{x} = \underset{M \times 1}{0}$$

with the optimization problem:

$$\min_x \|A \cdot x\|^2$$

and add constraint:  $\|x\|^2 = 1$  to avoid trivial solution

To do this, same as we did for heterogeneous linear systems, we will replace our homogeneous linear system with a least-squares optimization problem. This optimization problem still has  $x = 0$  as its solution.

To prevent this solution, we will add an extra constraint: We will search among only vectors that have length equal to 1, for the one that minimizes the least-squares loss function.



Solving the constrained linear least squares optimization

$$\min_{x: \|x\|^2=1} \|A \cdot x\|^2$$

Compute the *eigen vector* corresponding to the smallest eigen value  $A^T A$ :

Because this is a constrained optimization problem, we can no longer solve it simply by taking its derivative and setting it to zero.

It can be shown (we will not prove) that the solution to this problem is the eigenvector corresponding to the smallest eigenvalue of  $A^T A$

## Solving the constrained linear least squares optimization

$$\min_{x: \|x\|^2=1} \|A \cdot x\|^2$$

Compute the *singular value decomposition* (SVD) of  $A$ :

$$A = \overset{\substack{\text{left singular} \\ \text{vectors} \\ \text{(orthonormal)}}}{U} \cdot \overset{\substack{\text{singular} \\ \text{values} \\ \text{(diagonal)}}}{\Sigma} \cdot \overset{\substack{\text{right singular} \\ \text{vectors} \\ \text{(orthonormal)}}}{V}^T$$

$$M \times N \quad M \times M \quad M \times N \quad N \times N$$

Solution is equal to the column of  $V$  corresponding to the smallest singular value in  $\Sigma$ .

- Solution has unit norm because  $V$  is orthonormal

Alternatively we can use a linear algebra decomposition called the singular value decomposition, or SVD. Computing the SVD of  $A$  means that we write it as the product of three matrices: an  $M \times M$  matrix  $U$ , an  $M \times N$  matrix  $\Sigma$ , and an  $N \times N$  matrix  $V$ . The matrices  $U$  and  $V$  are orthonormal, meaning that all of their columns have unit norm. We call their columns the left and right singular vectors of  $A$ , respectively. The matrix  $\Sigma$  is diagonal, and we call its entries the singular values of  $A$ .

The SVD has many uses, such as determining the rank of a matrix, and so on. Here, we will use one of its properties, which you will prove in the tutorial : The solution to our constrained least-squares minimization problem is equal to the right singular vector corresponding to the smallest singular value in  $\Sigma$ . Note that, as the right singular vectors are columns of  $V$ , and  $V$  is orthonormal, this solution will automatically satisfy the unit norm constraint.

## Summary: least-squares solutions to linear systems

Heterogeneous linear system  $A \cdot x = b$

- compute pseudo inverse  $(A^T A)^{-1} A^T$
- compute solution as  $x = (A^T A)^{-1} A^T b$

Homogeneous linear system  $A \cdot x = 0$

- compute SVD  $A = U \cdot \Sigma \cdot V^T$
- compute solution as  $x$ : column of  $V$  corresponding to the smallest singular value in  $\Sigma$

So, here is a summary of how we will be solving linear systems: When the linear system is heterogeneous, meaning that its right hand side is nonzero, we can solve it by computing the pseudo-inverse, and multiplying it with the right hand side. When the system is homogeneous, meaning that the right hand-side is zero, we can solve it by computing the SVD of  $A$ , and taking the column of  $V$  corresponding to the smallest singular value in  $\Sigma$ . We will be encountering linear systems of this kind throughout this and the next four lectures, so it is important to be able to tell when they are heterogeneous versus homogeneous, and what algorithm we need to use to solve them.

.

## Summary: least-squares solutions to linear systems

Heterogeneous linear system  $A \cdot x = b$

- compute pseudo inverse  $(A^T A)^{-1} A^T$
- compute solution as  $x = (A^T A)^{-1} A^T b$

In Python, use:  
`numpy.linalg.solve`  
(generally a bad idea to  
compute matrix inverses)

Homogeneous linear system  $A \cdot x = 0$

- compute SVD  $A = U \cdot \Sigma \cdot V^T$
- compute solution as  $x$ : column of  $V$  corresponding to the smallest singular value in  $\Sigma$

In Python, use:  
`numpy.linalg.svd`

Numerical linear algebra packages, such as numpy in Python, give us robust and efficient routines for running these algorithms. You will not, and should not, try to write your own routine to compute the SVD, at least for the purposes of this course. As a note, in general it is a bad idea to explicitly compute inverses and pseudoinverses of matrices, because the inverse operation is very expensive and numerically unstable. Instead, numerical linear algebra packages provide functions directly for solving linear systems

## One more use of SVD: enforcing rank constraints

**Problem statement:** Given

- an  $M \times N$  matrix  $F$ ,
- an integer  $k < \min(M, N)$ ,

find the matrix  $F'$  of rank  $k$  that is closest to  $F$  in the least-squares sense

$$\min_{\substack{F' \\ \text{rank}(F')=k}} \|F - F'\|^2$$

**Solution:**

- compute the SVD  $F = U \cdot \Sigma \cdot V^T$
- form a diagonal matrix  $\Sigma'$  by replacing all but the  $k$  smallest singular values in  $\Sigma$  with 0.
- Compute the solution as  $F' = U \cdot \Sigma' \cdot V^T$

Before we finish this linear algebra detour, I want to mention one more property of SVD we will be using in later lectures. Consider the following problem: We have an  $M$  by  $N$  matrix  $F$ , and we want to approximate it, in the least squares sense, with a new matrix  $F'$ , of some lower rank  $k$ .

We can use the SVD of  $F$  to compute the new matrix  $F'$  as follows: Take the singular value matrix  $\Sigma$ , set all singular values except for the  $k$  largest to zero, then form  $F'$  by left-and-right multiplying the resulting matrix with  $U$  and  $V$ -transpose.

Determining unknown affine transformations

## Determining unknown affine transformations



Let's now put everything we learned about solving linear systems to use. We will do so for the following problem: We are given two images such as these, showing the same object under a different pose. One image is a warp of the other.

# Determining unknown affine transformations

Given a set of matched points...

$$\{p_i, p'_i\}$$

2D point in original image    2D point in transformed image

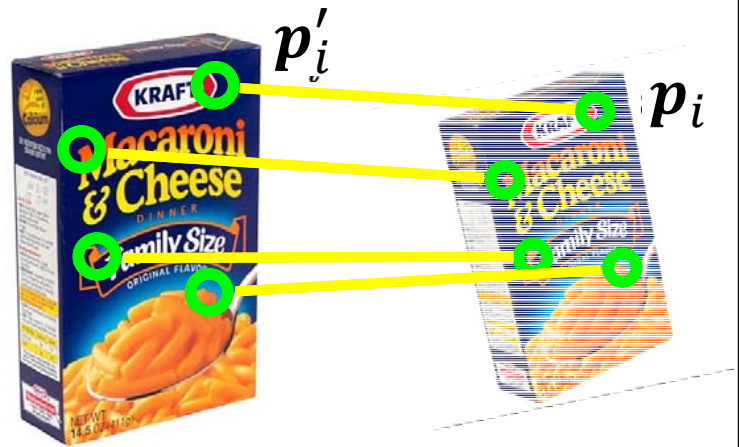
... and an affine transformation model...

$$p' = Mp$$

transformation matrix

... estimate the matrix

$$M = \begin{bmatrix} m_1 & m_2 & m_3 \\ m_4 & m_5 & m_6 \\ 0 & 0 & 1 \end{bmatrix}$$



Important: We will see a different procedure for determining unknown with homographies!

We run a feature detector, like those we discussed in the Feature Detection and Matching lecture, to create several pairs of matched points between the two images.

Given this set of matched points as input, we will now search for the *affine* transformation  $M$  that comes closest to mapping one set of points to the other. I want to emphasize here that we are limiting our search to affine transformations. We will see a different procedure for determining unknown homographies later in this lecture.



## Determining unknown transformations

Affine transformation: 
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} m_1 & m_2 & m_3 \\ m_4 & m_5 & m_6 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$
 Why can we drop the last line?

Vectorize transformation parameters:

Corresponding point pair 1:  $\begin{bmatrix} x'_1 \\ y'_1 \end{bmatrix} = \begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_1 & y_1 & 1 \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ m_5 \\ m_6 \end{bmatrix}$

Corresponding point pair 2:  $\begin{bmatrix} x'_2 \\ y'_2 \end{bmatrix} = \begin{bmatrix} x_2 & y_2 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_2 & y_2 & 1 \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ m_5 \\ m_6 \end{bmatrix}$

Stack equations from point correspondences:

Corresponding point pair j:  $\begin{bmatrix} x'_j \\ y'_j \end{bmatrix} = \begin{bmatrix} x_j & y_j & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_j & y_j & 1 \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ m_5 \\ m_6 \end{bmatrix}$

Notation in system form: 
$$\underbrace{\begin{bmatrix} x'_j \\ y'_j \end{bmatrix}}_{\mathbf{b}} = \underbrace{\begin{bmatrix} x_j & y_j & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_j & y_j & 1 \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ m_5 \\ m_6 \end{bmatrix}}_{\mathbf{x}}$$

To see how to determine the unknown matrix M, let's first write out the mapping relationship between the original and transformed point, for one of our matched point pairs.

Because our transformation is affine, we can drop the last row of the matrix, and also the last entry of the transformed homogeneous coordinates. We would not be able to do this, if we did not know that the transformation we are searching for is affine.

This is a linear relationship, that we can rearrange into a form where the unknown entries of the affine transform matrix are turned into a 6x1 vector. It's a simple linear algebra exercise to verify that the original and vectorized expressions are equivalent, and I invite you to try it out on your own.

All of this was for one pair of matched points. We can now take this expression, write it out for all of N point pairs we have available as input, and stack them together into an expression such as what is shown here.

Stack constraints from multiple point pairs

$$\underbrace{\begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_1 & y_1 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_N & y_N & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_N & y_N & 1 \end{bmatrix}}_A \underbrace{\begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ m_5 \\ m_6 \end{bmatrix}}_x = \underbrace{\begin{bmatrix} x'_1 \\ y'_1 \\ \vdots \\ x'_N \\ y'_N \end{bmatrix}}_b$$

We created a heterogeneous linear system!

Solve using pseudo-inverse!

$$\mathbf{x} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b}$$

Rearrange x into a 3x3 matrix M

In the end, we see that, through some linear algebra manipulation, we have used our input data and knowledge of the unknown transformation, to set up a heterogeneous linear system, where the unknowns are the entries of the affine transformation matrix we are looking to determine. Typically, we have multiple point pairs, meaning that N is pretty large. Therefore, this is an overdetermined heterogeneous linear system. The number of rows is much larger than the number of columns.

We can now make use of all the linear algebra we went through earlier: We can use the pseudo-inverse of matrix A, to solve this heterogeneous linear system, and determine the unknowns in vector x. We can then rearrange these unknowns into a 3x3 matrix, which gives us the affine transform we were looking to recover.

Remember again how we got here: we started from a problem setting involving some input geometric data and an unknown mapping between them. We did some algebraic manipulation to set up a linear system where the unknowns are the parameters of the unknown mapping. And, lastly, we used the linear algebra

tools from the previous slides to solve for our unknowns. We will be re-using this type of procedure again and again, both in this lecture and the next four.

One pair of points gives us two equations

$$\left\{ \begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_1 & y_1 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_N & y_N & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_N & y_N & 1 \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ m_5 \\ m_6 \end{bmatrix} = \begin{bmatrix} x'_1 \\ y'_1 \\ \vdots \\ x'_N \\ y'_N \end{bmatrix} \right.$$

$\underbrace{\hspace{15em}}$   
 $A$

$\underbrace{\hspace{5em}}$   
 $x$

$\underbrace{\hspace{5em}}$   
 $b$

We have six unknowns

We need *at least* three point pairs.

- Having more is fine, we are solving an overdetermined system.

Degenerate configurations: if all 3 points lie on the same line, there is no unique solution. Because the constraints are linearly dependent

Before we continue, it is worth taking a closer look at the dimensions of this system. The vector of unknowns  $x$  has size  $6 \times 1$ , which matches the degrees of freedom of the unknown affine transform.

Each pair of matched points gives us two rows in the matrix  $A$ , that is, two constraints on our unknowns.

Therefore, to be able to uniquely determine the unknown affine transformation, we need to have as input at least three pairs of matched points. Having fewer than three pairs of matched points will mean that we won't have a unique solution; in other words, there will be more than one affine transforms that exactly match the input to the target points. Having more than three pairs of matched points is fine: as we covered in the linear algebra slides, we will be simply solving an overdetermined system. In fact, typically it is better to have more constraints than needed, because then the least-squares solution helps deal with potential inaccuracies in the matched point pairs we are given as input.

Note that while most sets of 3 points define the affine transformation uniquely, there are degenerate cases. For example if we have exactly 3 points and all of them lie on the same line, the constraints at lines 5,6 of  $A$  are a linear combination of lines 1-4, so  $A$  is

not a full rank matrix and the solution is not unique.

## Classification of 2D transformations

| Name              | Matrix                                                                                                                                           | # D.O.F. |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| translation       | $\begin{bmatrix} 1 & 0 & t_1 \\ 0 & 1 & t_2 \\ 0 & 0 & 1 \end{bmatrix}$                                                                          | 2        |
| rigid (Euclidean) | $\begin{bmatrix} \cos \theta & -\sin \theta & t_1 \\ \sin \theta & \cos \theta & t_2 \\ 0 & 0 & 1 \end{bmatrix}$                                 | 3        |
| similarity        | $\begin{bmatrix} s \cdot \cos \theta & -s \cdot \sin \theta & t_1 \\ s \cdot \sin \theta & s \cdot \cos \theta & t_2 \\ 0 & 0 & 1 \end{bmatrix}$ | 4        |
| affine            | $\begin{bmatrix} r_1 & r_2 & r_3 \\ r_4 & r_5 & r_6 \\ 0 & 0 & 1 \end{bmatrix}$                                                                  | 6        |
| projective        | $\begin{bmatrix} r_1 & r_2 & r_3 \\ r_4 & r_5 & r_6 \\ r_7 & r_8 & r_9 \end{bmatrix}$                                                            | 8        |

Everything we discussed in this submodule has been about determining an unknown *affine* transformation. But remember, that we have another type of even more general transformations, that we named projective transformations or homographies.

## Computing with homographies

## Applying a homography

1. Convert to homogeneous coordinates:  $p = \begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow p = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$

What is the size of the homography matrix? ↘

2. Multiply by the homography matrix:  $P' = H \cdot P$

3. Convert back to heterogeneous coordinates:  $p' = \begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} \Rightarrow p' = \begin{bmatrix} x'/w' \\ y'/w' \end{bmatrix}$



## Applying a homography

1. Convert to homogeneous coordinates:

$$p = \begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow p = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

What is the size of the homography matrix? Answer: 3 x 3

2. Multiply by the homography matrix:

$$p' = H \cdot p$$

How many degrees of freedom does the homography matrix have?

3. Convert back to heterogeneous coordinates:

$$p' = \begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} \Rightarrow p' = \begin{bmatrix} x'/w' \\ y'/w' \end{bmatrix}$$

## Applying a homography

1. Convert to homogeneous coordinates:

$$p = \begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow p = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

What is the size of the homography matrix? Answer: 3 x 3

2. Multiply by the homography matrix:

$$p' = H \cdot p$$

How many degrees of freedom does the homography matrix have? Answer: 8

3. Convert back to heterogeneous coordinates:

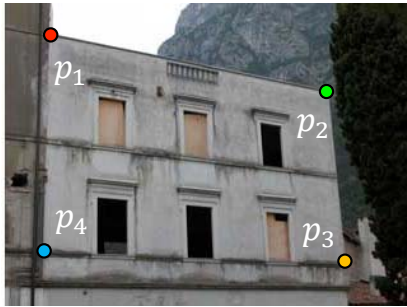
$$p' = \begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} \Rightarrow p' = \begin{bmatrix} x'/w' \\ y'/w' \end{bmatrix}$$

Determining homographies

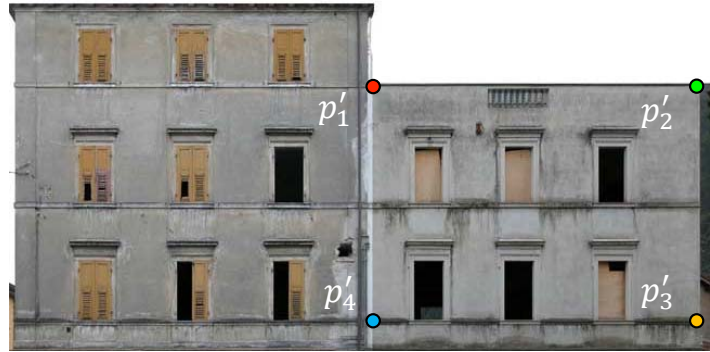
The direct linear transform (DLT)

## Determining unknown homographies

Start by creating point correspondences between two images



original image



target image

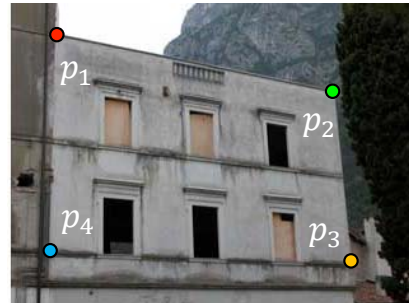
Let's see how we can determine an unknown homography, similar to what we saw in a previous part about affine transforms. Given the two images we want to map, we first need to find points in the two images that are the same. For that, for example, using the techniques from the Feature Detection and Matching lecture.

## Determining unknown homographies

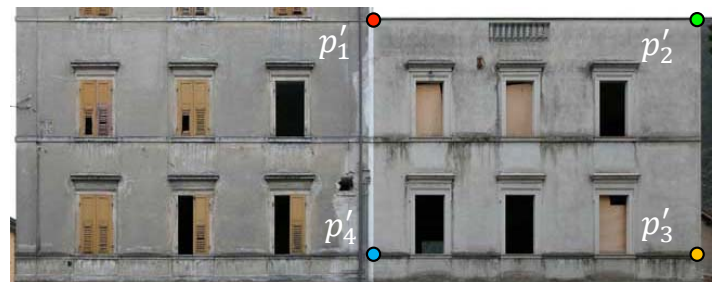
Given a set of matched points...

$$\{p_i, p'_i\}$$

2D point in original image    2D point in transformed image



original image



target image

Running a feature detector, we end up with a set of matched point pairs between the two images.

# Determining unknown homographies

Given a set of matched points...

$$\{p_i, p'_i\}$$

2D point in    2D point in  
original    transformed  
image    image

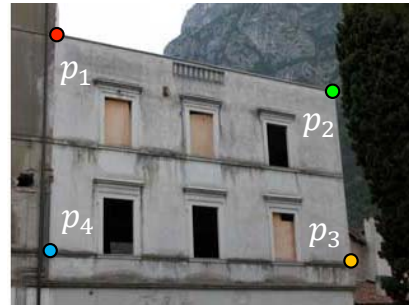
... and a projective transformation model...

$$p' = Hp$$

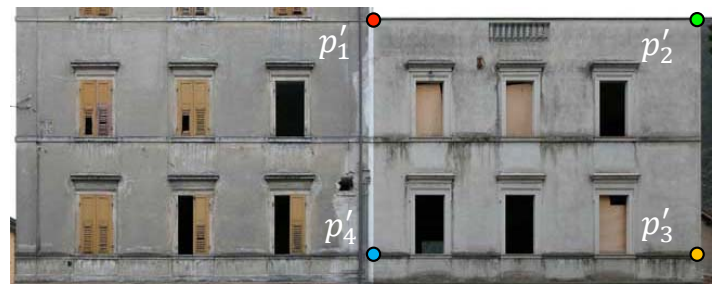
transformation  
matrix

... estimate the matrix

$$H = \begin{bmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & h_9 \end{bmatrix}$$



original  
image



target image

We then want to determine an unknown 3x3 matrix that, when used as a homography, can map, or at least come close to mapping, points from the original image to the transformed image. To do this, we will follow a procedure similar to what we did for determining an unknown affine transformation. Though note that the algorithm we will end up with will be very different from the affine case.

## Determining the homography matrix

Write out linear equation for each correspondence:

Note:  $\alpha$  is different for different point pairs  $p_i, p'_i$

$$p' = H \cdot p \quad \text{or} \quad \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \alpha \begin{bmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & h_9 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Expand matrix multiplication:

$$\begin{aligned} x' &= \alpha(h_1x + h_2y + h_3) \\ y' &= \alpha(h_4x + h_5y + h_6) \\ 1 &= \alpha(h_7x + h_8y + h_9) \end{aligned} \quad \Rightarrow x' = \frac{h_1x + h_2y + h_3}{h_7x + h_8y + h_9}$$

Divide out unknown scale factor:

$$\begin{aligned} x'(h_7x + h_8y + h_9) &= (h_1x + h_2y + h_3) \\ y'(h_7x + h_8y + h_9) &= (h_4x + h_5y + h_6) \end{aligned}$$

We start by writing out the linear mapping relationship for each pair of matched points. Note that, to write this expression, we need to convert our input points from heterogeneous to homogeneous coordinates. Unlike in the affine case, this means that, in addition to the unknown homography  $H$ , our mapping relationship will include some scalar  $\alpha$ . This is because, whereas in the affine transform we are guaranteed that the third coordinate does not change, and thus can be assumed to always be 1, in the homography case the third coordinate may change. The scalar  $\alpha$  corresponds to dividing out the unknown scaling of the third coordinate in the homogeneous vector of the mapped point.

We can now expand out the matrix multiplication into three equations. These equations are linear, but unfortunately they involve the additional unknown  $\alpha$  that we do not care about.

To deal with this, we can divide out the unknown scale factor, ending up with two linear relationships involving only the coordinates of the matched point pair, and the unknown entries of the homography matrix.

$$\begin{aligned}x'(h_7x + h_8y + h_9) &= (h_1x + h_2y + h_3) \\ y'(h_7x + h_8y + h_9) &= (h_4x + h_5y + h_6)\end{aligned}$$

Just rearrange the terms



$$\begin{aligned}h_7xx' + h_8yx' + h_9x' - h_1x - h_2y - h_3 &= 0 \\ h_7xy' + h_8yy' + h_9y' - h_4x - h_5y - h_6 &= 0\end{aligned}$$

We can take these relationships, and rearrange them a bit.



## Determining the homography matrix

Re-arrange terms:

$$h_7xx' + h_8yx' + h_9x' - h_1x - h_2y - h_3 = 0$$

$$h_7xy' + h_8yy' + h_9y' - h_4x - h_5y - h_6 = 0$$

Re-write in matrix form:

$$\mathbf{A}_i \mathbf{h} = \mathbf{0}$$

$$\mathbf{A}_i = \begin{bmatrix} -x & -y & -1 & 0 & 0 & 0 & xx' & yx' & x' \\ 0 & 0 & 0 & -x & -y & -1 & xy' & yy' & y' \end{bmatrix}_{2 \times 9}$$

$$\mathbf{h} = [h_1 \ h_2 \ h_3 \ h_4 \ h_5 \ h_6 \ h_7 \ h_8 \ h_9]^\top$$

As well as rewrite them in matrix form. In this form, the matrix A depends on only the known coordinates of the matched point pair, and the vector h is simply the nine entries of the unknown homography matrix, in vectorized form.

## Determining the homography matrix

Stack together constraints from multiple point correspondences:

$$\mathbf{A}\mathbf{h} = \mathbf{0}$$

$$\begin{bmatrix}
 -x & -y & -1 & 0 & 0 & 0 & xx' & yx' & x' \\
 0 & 0 & 0 & -x & -y & -1 & xy' & yy' & y' \\
 -x & -y & -1 & 0 & 0 & 0 & xx' & yx' & x' \\
 0 & 0 & 0 & -x & -y & -1 & xy' & yy' & y' \\
 \vdots & & & & & & & & \\
 -x & -y & -1 & 0 & 0 & 0 & xx' & yx' & x' \\
 0 & 0 & 0 & -x & -y & -1 & xy' & yy' & y'
 \end{bmatrix}
 \begin{bmatrix}
 h_1 \\
 h_2 \\
 h_3 \\
 h_4 \\
 h_5 \\
 h_6 \\
 h_7 \\
 h_8 \\
 h_9
 \end{bmatrix}
 =
 \begin{bmatrix}
 0 \\
 0 \\
 0 \\
 0 \\
 0 \\
 0 \\
 0 \\
 0 \\
 0
 \end{bmatrix}$$

$2N \times 9$        $9 \times 1$        $2N \times 1$

Homogeneous linear least squares problem

- Solve with SVD

This was all for one matched point pair. We can follow the same procedure for all point correspondences we have, and stack the resulting relationships into a linear system, where the matrix  $\mathbf{A}$  depends on only the known coordinates of all point correspondences, and the vector of unknowns  $\mathbf{h}$  has the homography matrix entries. Note that the linear system we arrived at is very different from the one we had for the affine transform: This system is homogeneous, whereas in the affine case the system was heterogeneous.

We only have one last step left: We need to solve this homogeneous linear system. As we learned in the linear algebra slides, we can do so using the SVD of  $\mathbf{A}$ .

## Solving for H using the direct linear transform (DLT)

Given  $\{p_i, p'_i\}$  solve for H such that  $p' = Hp$

1. For each correspondence, create 2x9 matrix  $A_i$

2. Concatenate into single  $2n \times 9$  matrix  $A$

3. Compute SVD of  $A = U\Sigma V^T$

4. Store singular vector of the smallest singular value  $h = v_{\hat{i}}$   This is a 9x1 vector

5. Reshape to get 3x3 matrix  $H$

The procedure we just went over goes by the name direct linear transform, and it is a fundamental algorithm of geometry-based computer vision. I summarize here the steps it involves.

## Determining the homography matrix

We need *at least* four point pairs.

- Having more is fine, we are solving an overdetermined system.

Degenerate configurations: if all 4 points lie on the same line, there is no unique solution.

One pair of points gives us two equations

$$\begin{bmatrix} -x & -y & -1 & 0 & 0 & 0 & xx' & yx' & x' \\ 0 & 0 & 0 & -x & -y & -1 & xy' & yy' & y' \\ -x & -y & -1 & 0 & 0 & 0 & xx' & yx' & x' \\ 0 & 0 & 0 & -x & -y & -1 & xy' & yy' & y' \\ \vdots & & & & & & & & \\ -x & -y & -1 & 0 & 0 & 0 & xx' & yx' & x' \\ 0 & 0 & 0 & -x & -y & -1 & xy' & yy' & y' \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \\ h_5 \\ h_6 \\ h_7 \\ h_8 \\ h_9 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

We have nine unknowns (matrix is determined *up to scale*)

Going back to our linear system, notice that our vector of unknowns has size by  $9 \times 1$ . Remember, though, that a homography matrix has only eight degrees of freedom, because homographies are defined up to scale. Remember, also, that, when solving homogeneous linear systems, we add a constraint that our solution vector  $h$  should have unit norm. This constraint removes one degree of freedom, bringing us back down to 8 as we expected from the properties of a homography.

Note also that one pair of matched points gives us two equations in the matrix  $A$  of our homogeneous system.

Therefore, we need at least *four* matched point pairs to uniquely determine an unknown homography. If we have fewer than 4, then there will be more than one homographies that exactly map input to target points. Having more than four matched point pairs is fine, as all it means is that we will be solving an overdetermined homogeneous system, which we know how to do using SVD. Notice that all these are exactly analogous to our discussion for determining an unknown affine transform, except that there we needed 3 point correspondences, whereas now we need four.

As before there are degenerate configurations, e.g. if all 4 points lie on a line there is no unique solution.

The proof here is not as simple as in the affine case, since the row of  $A$  are a non linear function of the matching point coordinates.

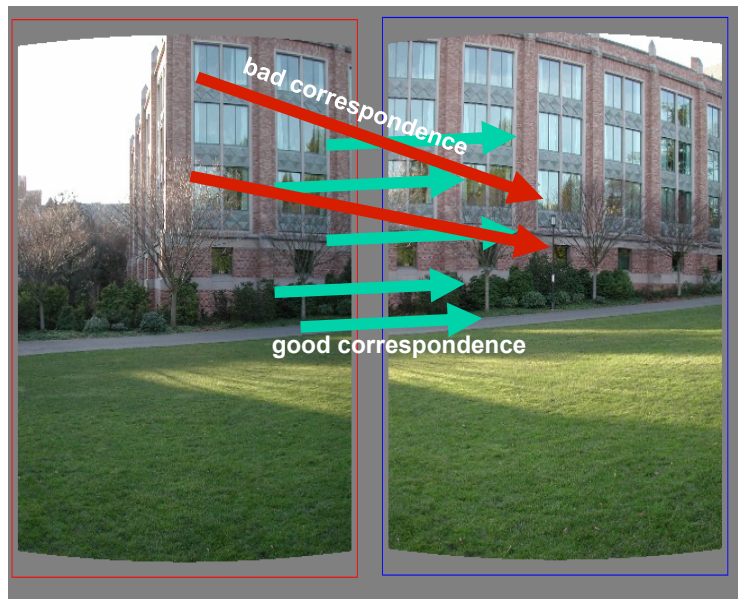
Random sample consensus (RANSAC)

# The image correspondence pipeline

1. Feature point detection
  - Detect corners using the Harris corner detector.
2. Feature point description
  - Describe features using the SIFT descriptor.
3. Feature matching

We have now seen two algorithms, one for determining unknown affine transformations, and another, the direct linear transform, for determining unknown projective (homography) transformations. Both of these algorithms require having as input point correspondences between the original and transformed image. Typically, we produce these using a 3-step procedure, as we described in the Feature Detection and Matching lecture: First, we run a feature detection algorithm, such as the Harris corner detector, on both the original and transformed image. Second, for each detected point, we compute a feature descriptor, such as the SIFT descriptor. Third and last, we use the descriptors to match detected points between the two images.

Unfortunately, this procedure is imperfect. While the majority of point correspondences it will produce will be correct correspondences, there will be a few that will be incorrect. We refer to those as outliers.

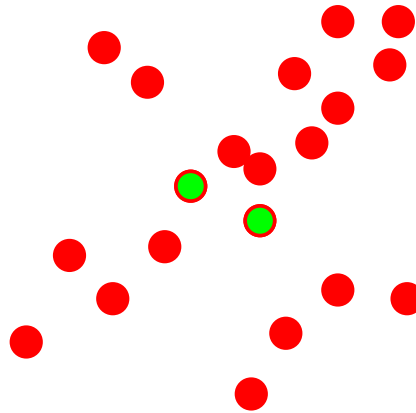


These outliers are common, for example, in scenes that have the same pattern repeating again and again, like the window corners in these two photographs. If we use this mixed bag of correspondences as input to our transformation estimation algorithms, we can potentially end up with very bad transformation estimates.

We can deal with this using an algorithm called Random Sample Consensus, or RANSAC for short. To get some intuition for how this algorithm works, we will see how we can use it to fit a line through a set of points, that include a few outliers.



I lwqj #b hv#  
+z wk#rxwduw,



### Algorithm:

#### 1. Sample (randomly) the number of points required to fit the model

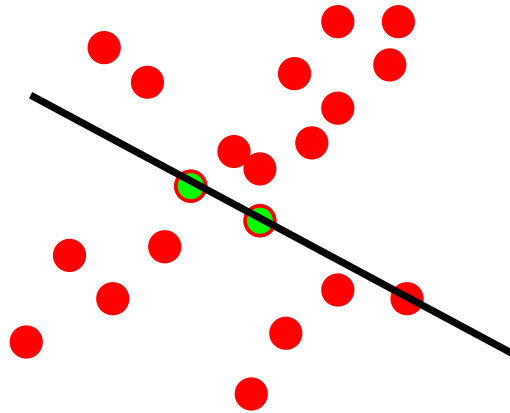
51 Vron#rup rgheddup hhuw#vbj#dp sdv#

61 Vfrun#e|kh#ufwrg# #qduw# wkq#b#uhvw#kuvkrog# #kh# rgho

Uhshdw#06#qwlkh#hvwp rghdw#rxqg# wk#lk#frqlghqfh

The algorithm runs in three steps. First, we *randomly* sample, out of our input data, the minimal number of points as we need to fit our model. In the case of line fitting, this is just two points.

I lwqj #bghv#  
+z wk#rxwduw,



### Algorithm:

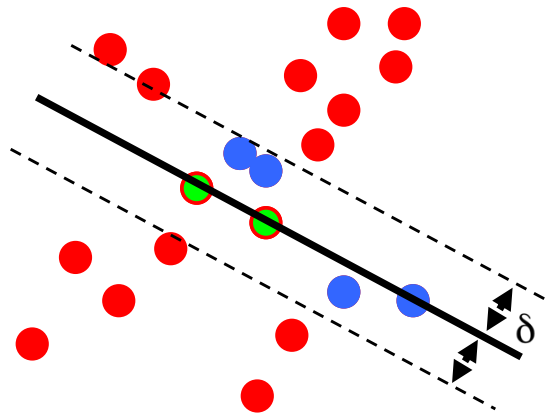
```
41 Vdp sduqgrp d ,#kh#xp ehur #srlbw#htxlng#r#lwk#p rgho
51 Solve for model parameters using samples
61 Vfruh#e|#kh#udfwrg# #gduw# wkq#l#uhvw#kuvkrog# #kh#p rgho

Uhshdw#l06#qwlwk#ehvw#p rghdw# #rxqg# wk#ljk#frqlghqfh
```

Second, we use *just* the sample points, to fit our model. In the case of line fitting, we just compute the line passing through the two points we sampled.

Ilwqj#bqhv#  
+z wk#rxvduw,

$$N_I = 6$$



### Algorithm:

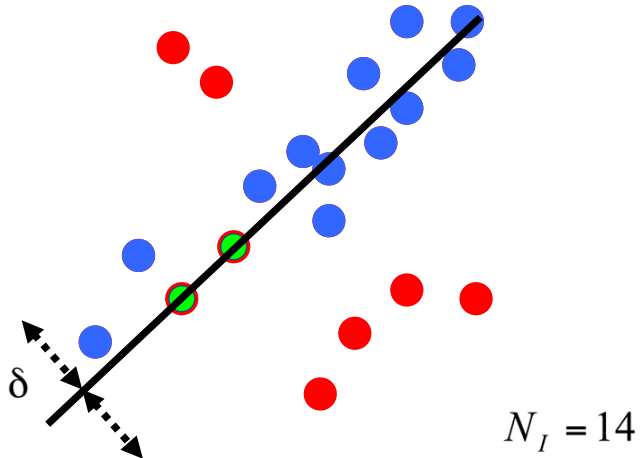
```
41 Vdp sduqgrp d, #kh#xp ehur#srlw#htxlng#r#lk#p rgho
51 Vron#rup rgho#dup hhuw#vbj#dp sdv#
```

### 3. Score by the fraction of inliers within a preset threshold of the model

```
Uhshdw#06#qwlkh#ehw#p rgho#r#xqg# wk#lk#Frqlghqfh
```

Third, we score the model we fitted, by computing how many of the other points in the dataset are approximated well by it. In the case of line fitting, this means computing how many other points in our dataset are within some distance delta from the line we fitted. Points that are approximated well form an inlier set for this model proposal, and all other points are excluded as outliers.

Ilwlgj #bghv#  
+z lk#rxvduw,



### Algorithm:

```

41 Vdp sduqgrp d, #kh#xp ehur #srlbw#htxlngw #lk#p rgho
51 Vroy#rup rgho#dudp hhuw#kvlgj #dp sdv#
61 Vfruh #kh#udfwrg #gduwz lk#b#uhvw#kuvkrog #kh#p rgho

```

**Repeat 1-3 until the best model is found with high confidence**

RANSAC then runs by iterating through these steps multiple times, and at each iteration we keep track of the model that has the largest inlier set among all we have seen so far. At the end, we can use the largest inlier set to fit a model through just those points, excluding all outliers.

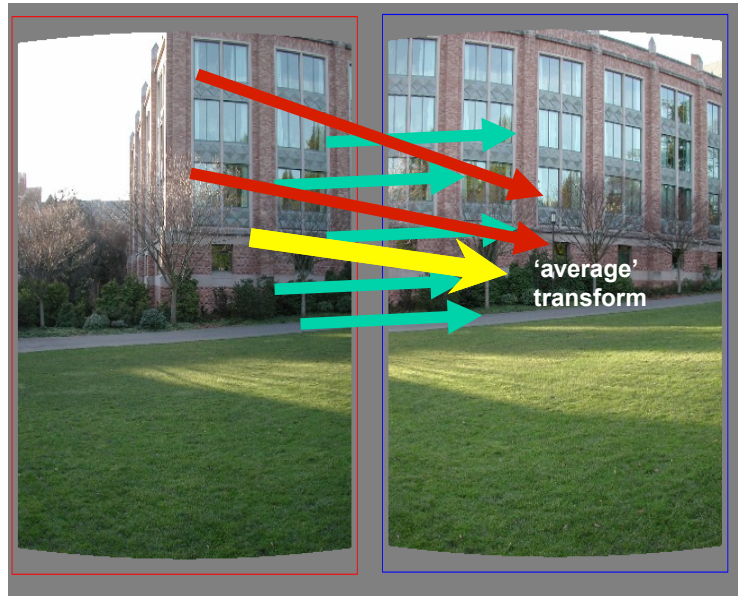
Matched points will usually contain bad correspondences



Let's look at an example of this for image transformations.

The computed correspondences that have some outliers.

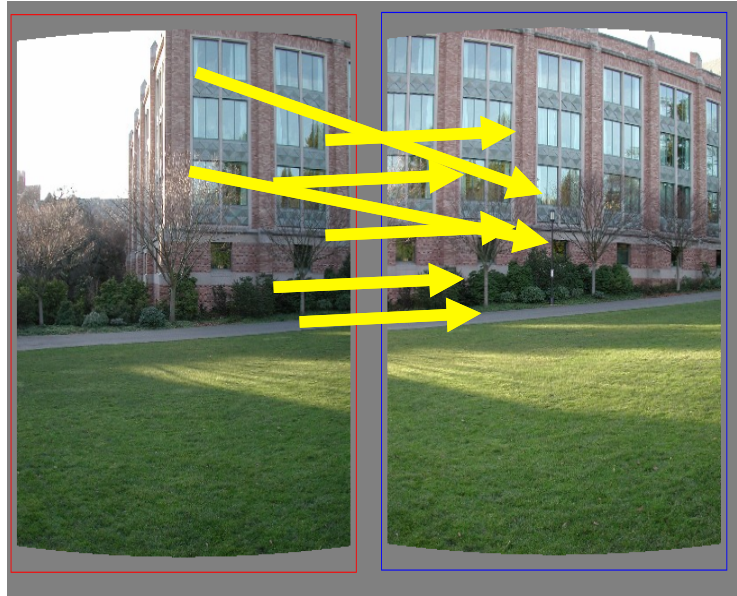
linear least squares will find the 'average' transform



solution is corrupted by bad correspondences

If we compute a translation transformation by using all of the matched pairs, we can end up with an “average” transform that is very heavily affected by the outliers, and therefore is quite inaccurate.

Use RANSAC

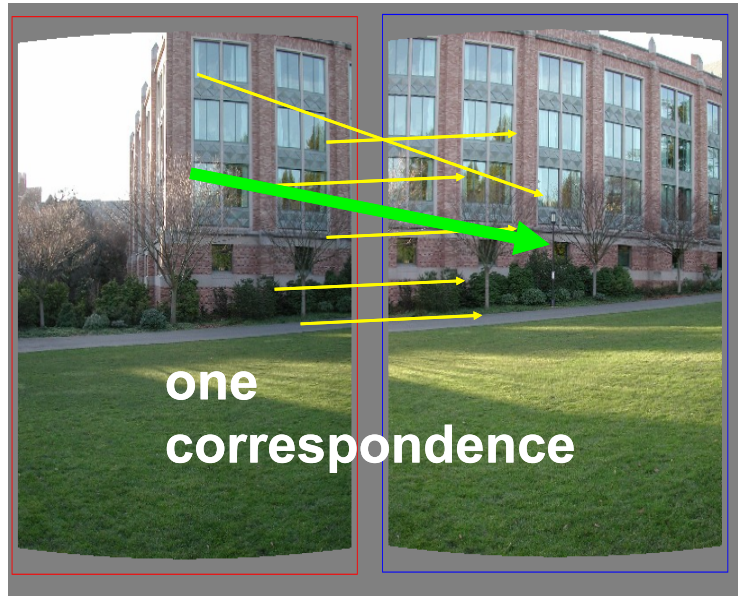


How many correspondences do we need for finding a translation model?

Let's now do the same using RANSAC.

To fit a translation transformation, we only need one point correspondence: Remember, a translation transformation has 2 degrees of freedom, and one point correspondence gives us two constraints on these 2 degrees of freedom, one for x and one for y coordinates.

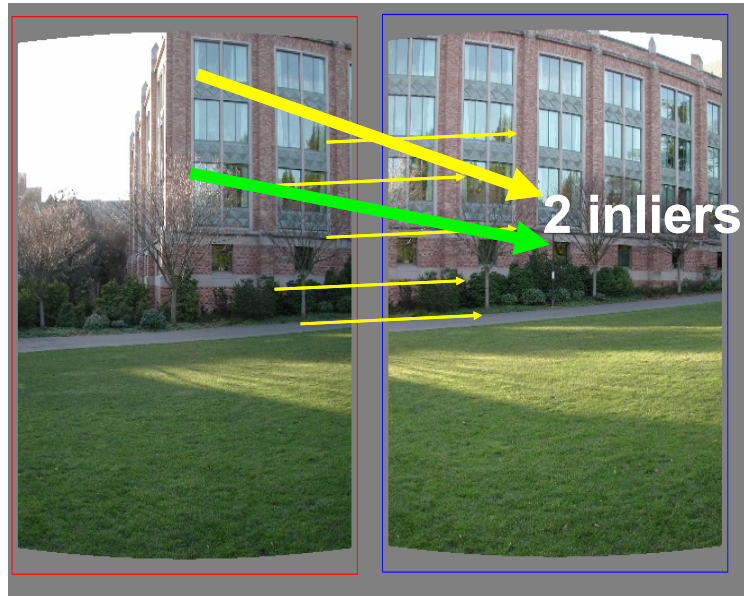
Pick one correspondence, count inliers



We randomly sample one correspondence, and compute the associated translation, shown here in green.

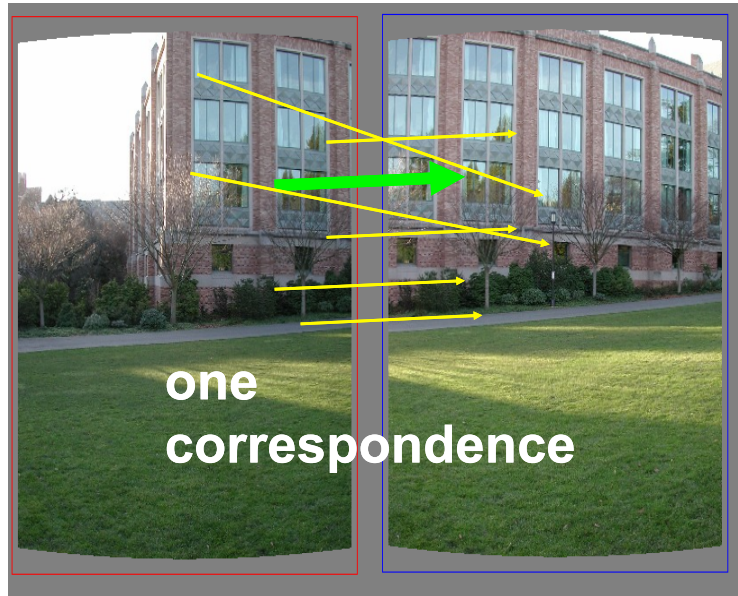


Pick one correspondence, count inliers



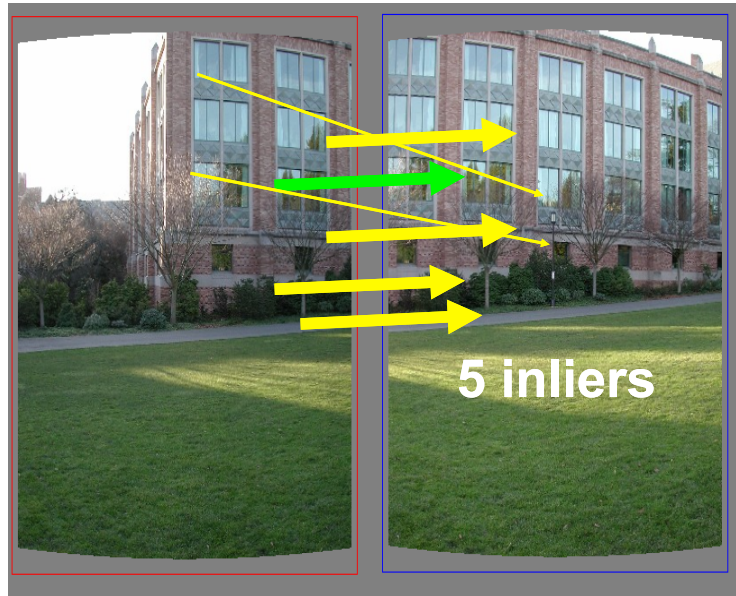
We count how many of the total number of correspondences will be inliers, in this case just two.

Pick one correspondence, count inliers



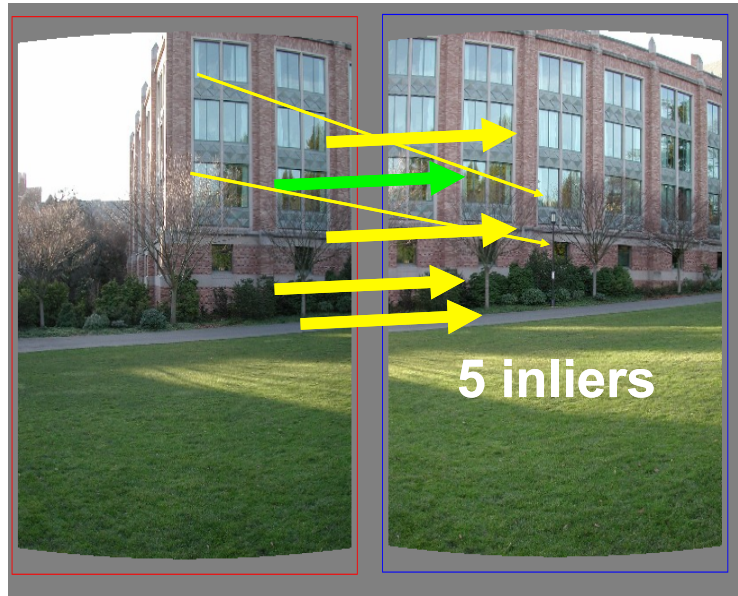
Now let's sample another correspondence, and again fit a translation, shown in green.

Pick one correspondence, count inliers



Now we have 5 inliers, which is greater than the number we had before.

Pick one correspondence, count inliers



Pick the model with the highest number of inliers!

Repeating this procedure many times, we will end up with a largest set of inliers that will match the one shown here. Notice that the set correctly excludes the two inaccurate correspondences. We can then use all the inlier correspondences, to accurately fit a translation transformation.

## Homography estimation using RANSAC

- RANSAC loop
    1. Get four point correspondences (randomly)
    2. Compute  $H$  using DLT
    3. Count inliers
    4. Keep  $H$  if largest number of inliers
  - Recompute  $H$  using all inliers
- 

We can apply this RANSAC procedure to robustly estimate a homography estimation. We only need to change two things: First, at every iteration we need to randomly sample four point correspondences, rather than one. This is because, as we saw before, we need four point correspondences to uniquely determine a homography. Second, once we sample these four point correspondences, we compute a homography from them using the DLT algorithm we saw before. These steps are summarized here. I will close this discussion by mentioning that RANSAC is an algorithm that you can use any time you need to perform some form of fitting task on data that can include outliers. It is particularly popular in computer vision, because it has a track record of strong empirical performance across a large variety of applications.

Forward and inverse warping

# Implementing image warps

Suppose we have two images and a warp transform  $T$ .

- How do we form the transformed image?

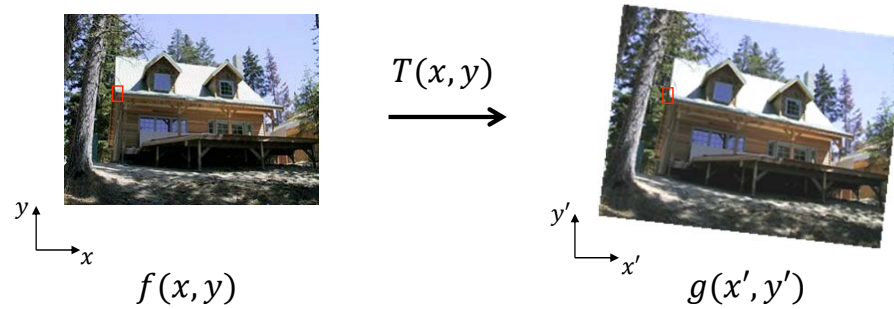


So far, we have seen how to decide what type of transformation to use, as well as how to determine the exact parameters of a transformation, given some input data such as point correspondences between an original and a transformed image. The last topic we are going to cover in this module is: Let's say that I have the original image, and a known warp transform  $T$ . How exactly do I form the transformed image?

# Forward warping

Suppose we have two images and a warp transform  $T$ .

- How do we form the transformed image?



Approach 1 (forward warping):

- Send intensities  $f(x, y)$  in first image to location  $(x', y') = T(x, y)$  in the second image

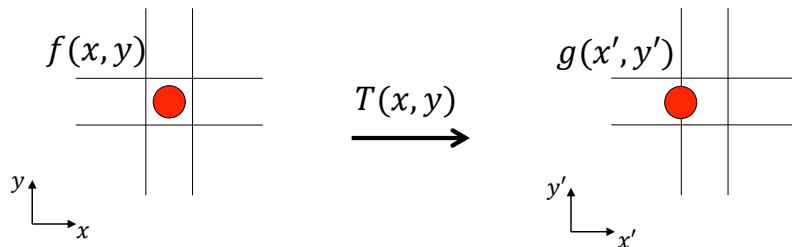
Broadly speaking, there are two approaches for implementing a warping transform. The first is called forward warping. The way it works is that, we use a double for loop to go over every pixel  $x$ - $y$  in the original image, and use the transform  $T$  to find the location  $x'$ - $y'$  it maps to in the second image.



# Forward warping

Pixels may end up between two points

- How do we determine the intensity of each point?

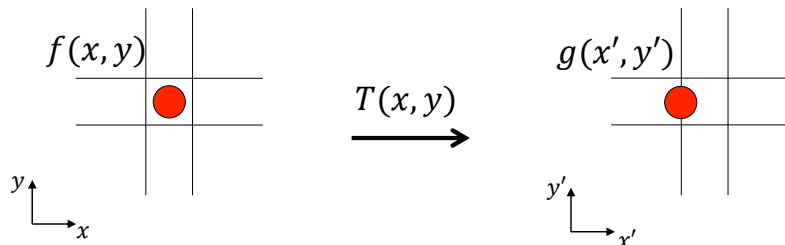


Remember, however, that in real images, we do not have continuous coordinates  $x$ - $y$ . We have a discrete grid of locations, corresponding to the pixels of the image. The issue we run into using forward warping is that a pixel  $x$ - $y$  in the original image can be mapped at a location  $x'$ - $y'$  between two pixels in the transformed image. We then need to decide exactly how to determine the intensity in the transformed image.

# Forward warping

Pixels may end up between two points

- How do we determine the intensity of each point?
- ✓ We distribute intensity among neighboring pixels  $(x', y')$  ("splatting")



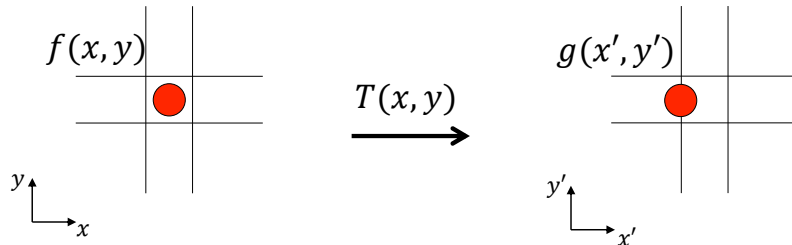
- What if a pixel  $(x', y')$  receives intensity from more than one pixels  $(x, y)$ ?

What we can do is *distribute* the intensity of the pixel in the original image among all pixels neighboring the location  $x'-y'$  in the transformed image. This operation is called splatting. Consider now a pixel in the transformed image that, through splatting, receives intensity contributions from more than one pixels in the original image.

# Forward warping

Pixels may end up between two points

- How do we determine the intensity of each point?
- ✓ We distribute intensity among neighboring pixels  $(x', y')$  ("splatting")



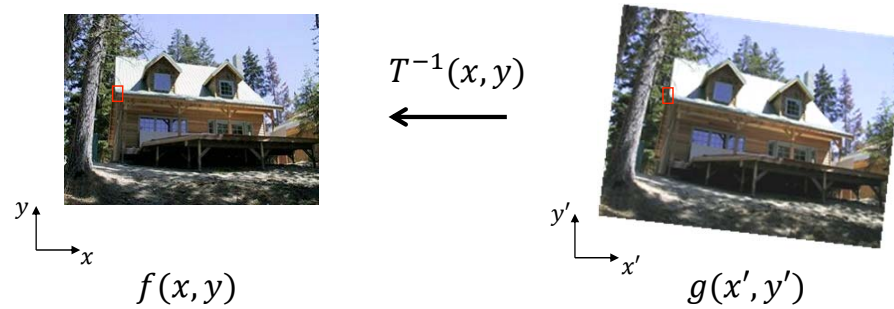
- What if a pixel  $(x', y')$  receives intensity from more than one pixels  $(x, y)$ ?
- ✓ We average their intensity contributions.

All we have to do to determine the intensity of the pixel in the transformed image is average the multiple contributions it receives through splatting.

# Backward warping

Suppose we have two images and a warp transform  $T$ .

- How do we form the transformed image?



Approach 2 (backward warping):

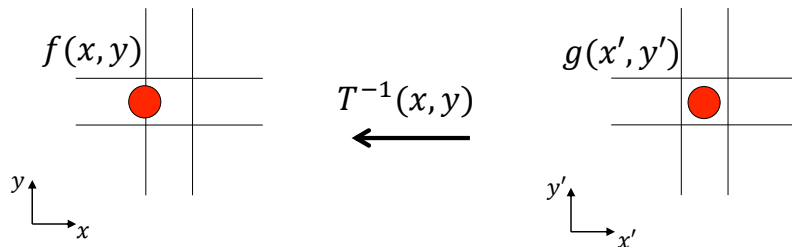
- Get intensities  $g(x', y')$  in the second image from point  $(x, y) = T^{-1}(x', y')$  in first image

The second approach for implementing a warping transform is called backward (or inverse) warping. The way this works is that we now use a double for loop to go over every pixel  $x'-y'$  in the transformed image. For each such pixel, we use the *inverse* transform  $T^{-1}$  to figure out what point in the original image it corresponds to.

## Backward warping

Pixel may come from between two points

- How do we determine its intensity?



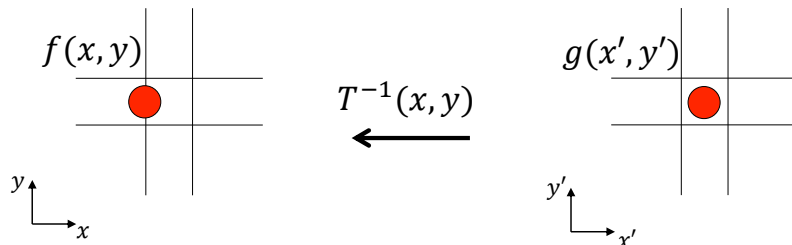
The problem we run into in this case is that a pixel  $x'-y'$  in the transformed image may be mapped to a point  $x-y$  in the original image that is between multiple pixels. We need to determine what intensity to use at that point.

## Backward warping

Pixel may come from between two points

- How do we determine its intensity?

✓ Use interpolation

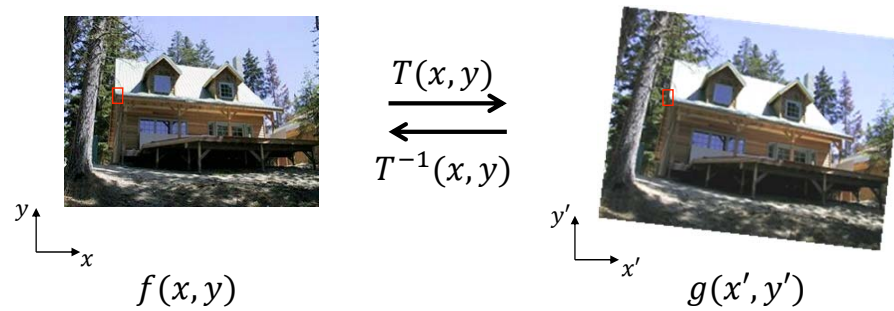


This is not such a big problem though. We can use interpolation procedures, such as bilinear interpolation, to determine the intensity of the original image at point  $x$ - $y$  as a weighted average of the intensities of all pixels adjacent to that point.

## Forward versus backward warping

Suppose we have two images and a warp transform  $T$ .

- How do we form the transformed image?

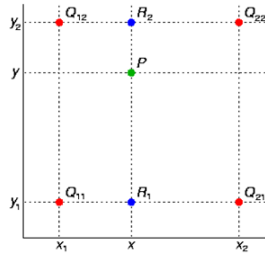


- Backward warping eliminates holes in target image
- Forward warping does not require existence of inverse transform

Usually  
recommended

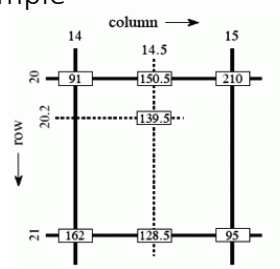
Compared to forward warping, inverse warping has the advantage that, by design, it fills every pixel in the transformed image. By contrast, forward warping can produce holes in the transformed image. The downside of inverse warping is that it requires knowing the inverse transform. For the linear transformations we have covered in this module, this is not a big problem: For an affine transform or a homography, the inverse transformation corresponds to simply using the inverse transformation matrix, which we can readily compute using linear algebra.

# Bilinear interpolation



1. Interpolate to find  $R_2$
2. Interpolate to find  $R_1$
3. Interpolate to find  $P$

Grayscale example



In matrix form (with adjusted coordinates)

$$f(x, y) \approx \begin{bmatrix} 1-x & x \end{bmatrix} \begin{bmatrix} f(0, 0) & f(0, 1) \\ f(1, 0) & f(1, 1) \end{bmatrix} \begin{bmatrix} 1-y \\ y \end{bmatrix}.$$

Matlab:  
call `interp2`

Python:  
SciPy `interp2d()`

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.interp2d.html>

```
from scipy.interpolate import interp2d
f = interp2d(x, y, z, kind='cubic')
znew = f(xnew, ynew)
```



# References

Basic reading:

- Szeliski textbook, Section 3.6., 6.1

Additional reading:

- Hartley and Zisserman, "Multiple View Geometry in Computer Vision," Cambridge University Press 2004.  
a comprehensive treatment of all aspects of projective geometry relating to computer vision,  
Sections 2 and 4 in particular discuss everything about homography estimation
- Richter-Gebert, "Perspectives on projective geometry," Springer 2011.  
a beautiful, thorough, and very accessible mathematics textbook on projective geometry.